
Continuous Latent Contexts Enable Efficient Online Learning in Transformers

Emile Anand

Georgia Institute of Technology
emile@gatech.edu

Abdullah Ateyeh

University of California, Berkeley
abdullah_ateyeh@berkeley.edu

Xinyuan Cao

Georgia Institute of Technology
xcao78@gatech.edu

Max Dabagia

Columbia University
rzd2155@columbia.edu

Abstract

Large language models (LLMs) exhibit a strong capacity for in-context learning: Given labeled examples, they can generate good predictions without parameter updates. However, many interactive settings go beyond static prediction to online decision-making, in which effective behavior demands adaptation over long multi-turn horizons in response to feedback, and efficient algorithms in these domains must use compact representations of what they have learned. Recently, continuous transformer architectures with latent chain of thought have shown promise for offline iterative tasks such as directed graph-reachability. Motivated by this, we study whether continuous latent context tokens equip transformers to more effectively realize online learning. We give explicit constructions of constant-depth transformers that implement two foundational online decision-making procedures – the weighted majority algorithm and Q -learning – by storing their algorithmic state as linear combinations of feature embeddings, using a small number of latent context tokens. We further train a small GPT-2-style transformer with latent contexts using a multi-curriculum objective that does not directly supervise the latent states. On long synthetic online prediction sequences, this model outperforms larger and more complex LLMs, including Qwen-3-14B and DeepSeek-V3. Our results suggest that continuous latent contexts provide a simple and effective persistent state for transformers to implement online learning algorithms.

1 Introduction

Large language models (LLMs), built on the transformer architecture [Vaswani et al., 2017], have demonstrated a striking ability to perform in-context learning: Given a sequence of examples, they can infer a useful predictor without updating their parameters. This ability is especially relevant in interactive settings, where a model receives (often, implicit) feedback over multiple turns and must adapt its future behavior accordingly [Lin et al., 2024b]. For example, in a multi-turn conversation, the user’s responses provide implicit feedback on which interpretations, plans, or styles are effective. In turn, a helpful conversational agent might use this feedback to improve user satisfaction.

This perspective raises a fundamental question: *Can transformers implement long-horizon online learning algorithms within their context?* In online decision-making, the learner acts sequentially, receives feedback, and records some information to inform future decisions (often conceived as maintaining an algorithmic state). Decisions are often irrevocable, and performance is measured against adaptive or hindsight targets such as the best expert, the best policy, or the optimal value function [Hazan, 2023, Kalai and Vempala, 2005, Cohen and Hazan, 2015]. In the case of a

conversational agent, online decision-making can support reasoning, whether incorporating user feedback, directing lines of reasoning, or allocating resources, in its broader goal of improving user satisfaction. Therefore, a good online reasoning model should be a good online decision making model [Akyürek et al., 2023], and good online decisions should follow from effective reasoning.

In standard chain-of-thought prompting intermediate reasoning is represented by discrete tokens, which can become very long [Wei et al., 2023]. A recent development in latent reasoning is the transformer architecture with continuous chain-of-thought (COCONUT) [Hao et al., 2025], where the continuous states can carry information in “superpositions” of vocabulary tokens that might be lost by discretization. It was recently shown that COCONUT admits a constant-depth transformer to solve the directed graph-reachability problem on an n -vertex graph in $O(D)$ steps (where $D < n$ is the diameter of the graph) [Zhu et al., 2025c], whereas discrete chain-of-thought needs $O(n^2)$ autoregressive steps [Merrill and Sabharwal, 2024]). This is a clear example of how continuous latent representations can serve as a compact representation of context, such as algorithmic state.

Online learning is a natural setting in which to explore similar ideas. For online decision-making, the ability to extract and maintain compactly the information necessary for future decisions is central [Anand and Karmarkar, 2026, Yang et al., 2021, Anand et al., 2025]. Classical online algorithms such as weighted majority, online gradient methods, and tabular reinforcement-learning methods all can be abstracted as compressing history into a low-dimensional state and updating that state incrementally as new feedback arrives [Hazan et al., 2017, Kalai and Vempala, 2005, Zinkevich, 2003, Simchowitz et al., 2020, Flaxman et al., 2004, Lin et al., 2023, 2024a]. In the weighted majority algorithm this state is a set of expert weights which are updated via multiplicative weights, while in tabular reinforcement learning the state is a value estimate for each state which are updated by various online strategies for dynamic programming, including SARSA and Q -learning. This notion of an online algorithmic state matches the computational role of latent context: to summarize the relevant history, be read by later attention heads, and be updated to incorporate new information.

1.1 Contributions

In this paper, we show that a small number of continuous context tokens suffices to allow transformers to perform well on fundamental online decision-making problems. Specifically:

- We give explicit transformer constructions for multiplicative weights (Littlestone and Warmuth [1994]) in the experts prediction setting, using a continuous latent token to store expert log-weights as a superposition, and tabular Q -learning (Watkins and Dayan [1992]), using continuous event tokens which encode action-indexed rows of the Q -table as a superposition,
- We train a transformer with continuous latent contexts using a multi-curriculum strategy and loss function that is agnostic to the latent contexts, and we show comparable performance against the corresponding classical online-learning algorithms,
- We evaluate Qwen-3-14B (40 layers, 14.8B parameters) [Yang et al., 2025] and DeepSeek-V3 (61 layers, 671B parameters) DeepSeek-AI et al. [2025] on online prediction tasks in a synthetic setting and more realistic weather-forecasting narrative. In these long-sequence online decision-making tasks, we show that our simpler transformer (4 layers, 200K parameters) is more effective than these larger LLMs, highlighting the value of continuous latent contexts.

Taken together, our results support an algorithmic interpretation of continuous latent reasoning: that transformers with continuous latent contexts can use their context as a persistent online-learning state, enabling them to implement online-learning algorithms,

1.2 Related Works

Expressiveness of transformers. There is a substantial body of work studying the expressivity of transformers. Most relevant to this paper are those which examine how some form of recurrence can improve expressiveness (most notably in chain-of-thought). For example, Chen et al. [2024] studies low depth transformers for arbitrary problems and shows that for any L there is some problem that a CoT transformer can solve in L layers that a non-CoT transformer will need $L + 1$ layers for. Wei et al. [2023], Feng et al. [2023] shows that constant-depth transformers with CoT can solve certain P-complete problems. Chen et al. [2024] provides constructions for certain constant-depth transformers for each problem in $P \setminus \text{poly}$ with CoT. There is also work that shows that logarithmically many steps

of CoT in input length can expand the upper bound of constant-depth transformer expressiveness from TC^0 to L , while a linear number of steps can further expand the upper bound to NC^1 complete [Merrill and Sabharwal, 2025, Li et al., 2024, Chiang et al., 2023, Strobl et al., 2024, Li et al., 2025]. However, these typically focus on discrete CoTs whereas we look at the continuous version.

Reasoning in the context of optimization. Many reasoning tasks have an optimization character, and indeed reasoning itself in LLMs has been studied as an exploration/exploitation tradeoff [Lin et al., 2024a, Xie et al., 2024, Anand and Liaw, 2025]. It has previously been shown that transformers can learn to realize gradient descent [Gatmiry et al., 2024, Huang et al., 2025, Innocenti and Achour, 2025, Kim et al., 2026, Chan et al., 2025, Wei et al., 2026, Laskin et al., 2022, Fu et al., 2024], which are important precursors to this work, where we apply similar ideas to the latent reasoning setting.

Latent reasoning. Prior work shows that LLMs contain structured internal representations, including linear encodings of space and time [Kong et al., 2024, Gurnee and Tegmark, 2023] and compact vectors representing in-context functions [Todd et al., 2024]. Recent methods make such hidden computation more explicit through pause [Goyal et al., 2024], planning [Wang et al., 2024b], and filler tokens that enable hidden computation without text [Pfau et al., 2024, Ramji et al., 2026, Liu et al., 2024, Wang et al., 2026, Cao et al., 2026]. Here, we study transformers which autoregressively generate their own future inputs (as in chain-of-thought), but without discretizing into discrete tokens. Building models which use this reasoning space effectively is not trivial, since the scaffolding provided by pre-training on natural language does not necessarily benefit. Various approaches have had some success: Amos et al. [2026] proposed an architecture based around continuous tokens produced as a supervised summary of a natural language chain-of-thought. The Huggin architecture contains directly recurrent modules which may iterated arbitrarily for latent reasoning [Geiping et al., 2025], and these latent reasoning chains have been shown to encode information about predictions [Du et al., 2026]. COCONUT allows for fully continuous chains-of-thought [Hao et al., 2025], warm-starting training with a curriculum that first trains reasoning using discrete chains and then relaxes the constraint. Follow-up theory explained its advantage via superposition of multiple reasoning paths [Zhu et al., 2025b,a]; the approach we consider here was most directly inspired by COCONUT.

2 Preliminaries

Basic notations. For any integer $N > 0$, we use $[N]$ to denote the set $\{1, 2, \dots, N\}$. For any finite set $\mathcal{X}$, let $|\mathcal{X}|$ denote the cardinality of $\mathcal{X}$. Let $\mathbb{R}$ be the set of real numbers. We use $\mathbb{1}\{\cdot\}$ to denote the indicator function. We use lower-case bold letters (e.g., $\mathbf{x}, \theta$) and upper-case bold letters (e.g., $\mathbf{W}, \mathbf{U}$) to denote vectors and matrices, respectively. In particular, we use $\mathbf{I}_d$ to denote a $d \times d$ identity matrix, use $\mathbf{0}_{m \times n}$ (or $\mathbf{0}_m$) to denote an $m \times n$ zero matrix (or an m -dimensional zero vector), and use $\mathbf{e}_i$ to denote a one-hot vector of which the i -th entry is one and other entries are all zero, where the dimension of $\mathbf{e}_i$ can be inferred from the context. We also use $\|\cdot\|_\infty$ and $\|\cdot\|_2$ to represent ℓ_∞ and ℓ_2 norm, respectively. For vectors $\mathbf{u} \in \mathbb{R}^m$ and $\mathbf{v} \in \mathbb{R}^n$, let $\mathbf{u} \otimes \mathbf{v} = \mathbf{u}\mathbf{v}^\top \in \mathbb{R}^{m \times n}$ denote their inner product. Finally, for any vector $\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^d$, we define the softmax function $\text{SoftMax} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ as $\text{SoftMax}(\mathbf{x})_i = \exp(x_i) / (\sum_{j=1}^d \exp(x_j))$.

Tokens and Embeddings. Fix a vocabulary $\mathcal{V} = [V]$. Each token $v \in \mathcal{V}$ has an embedding $\mathbf{u}_v \in \mathbb{R}^d$, whose dimension $d = k d_{\text{TE}} + d_{\text{PE}}$ decomposes as k token-embedding blocks and one positional block. For a vector $\mathbf{x} = (x_1, x_2, \dots, x_d)^\top \in \mathbb{R}^d$, we define $\text{id}(\mathbf{x}) = (x_1, \dots, x_{d_{\text{TE}}})^\top$, $\text{buf}_j(\mathbf{x}) = (x_{j \cdot d_{\text{TE}} + 1}, \dots, x_{(j+1) \cdot d_{\text{TE}}})^\top$ for $j \in [k]$, and $\text{pos}(\mathbf{x}) = (x_{k \cdot d_{\text{TE}} + 1}, \dots, x_d)$. Let $\tilde{\mathbf{u}}_v = \text{id}(\mathbf{u}_v) \in \mathbb{R}^{d_{\text{TE}}}$ for $v \in \mathcal{V}$. Next, let $\mathbf{U} = [\tilde{\mathbf{u}}_1, \tilde{\mathbf{u}}_2, \dots, \tilde{\mathbf{u}}_V] \in \mathbb{R}^{d_{\text{TE}} \times V}$ and assume token orthonormality, $\mathbf{U}^\top \mathbf{U} = \mathbf{I}_V$.

Algorithm 1 Decoder-only Causal Transformer (TF) with Chain of Continuous Thought

Require: input embeddings $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_t)$, learned absolute positional encoding parameter θ_{PE}

$\mathbf{h}_i^{(0)} \leftarrow \mathbf{x}_i + \text{PosEncode}_{\theta_{\text{PE}}}(i)$, for all $i \in [t]$

for $\ell = 0$ to $L - 1$ **do**

$h^{\ell+0.5} \leftarrow h^\ell + \sum_{m=1}^{H_\ell} \text{Attn}_{\theta_{\text{Attn}}^{(\ell, m)}}(h^\ell)$

$h^{\ell+1} \leftarrow \text{LayerNorm}(\text{MLP}_{\theta_{\text{MLP}}^\ell}(h^{\ell+0.5}))$

Output: Embedding of the last layer at the last position $h_t^{(L)} =: \mathbf{x}_{t+1}$.

Transformer architectures. An L -layer autoregressive transformer receives a sequence of input embeddings $\mathbf{h} = \mathbf{h}_{[t]} = (\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_t) \in \mathbb{R}^{d \times t}$ and outputs $\text{TF}_\theta(h) \in \mathbb{R}^d$ where $\text{TF}_\theta(\cdot)$ is defined in Algorithm 1. Let $\mathbf{W}_O \in \mathbb{R}^{V \times d}$ be the decoding matrix. A traditional decoder will sample the next token $v_{t+1} \sim \text{SoftMax}(\mathbf{W}_O \text{TF}_\theta(h))$ and append its token embedding in position $t+1$, i.e. $h_{t+1} = u_{v_{t+1}}$, to autoregressively generate subsequent outputs. When using the chain of continuous thought, one skips the sampling step and directly appends the output of the transformer as the input embedding of the next position, i.e. $h_{t+1} = \text{TF}_\theta(h)$. θ_{PE} denotes the learned absolute positional encoding, and each attention layer $\ell \in [L]$ has H_ℓ heads $\{\theta_{\text{Attn}}^{(\ell, m)}\}_{m=0}^{H_\ell-1}$.

Algorithm 2 Causal self-attention (Attn)

Require: Input $h = (h_1, \dots, h_t)$ and $\theta_{\text{Attn}} = (Q, K, V, O)$

Compute the queries $q_i \leftarrow Qh_i$, keys $k_i \leftarrow Kh_i$ and values $v_i \leftarrow Vh_i$, for all $i \in [t]$

for $i = 1, \dots, t$ **do**

$s_i \leftarrow \text{SoftMax}(\langle q_i, k_1 \rangle, \dots, \langle q_i, k_t \rangle)$

$h_i^{\text{Attn}} \leftarrow O \sum_{j=1}^i s_{i,j} v_j$

Output: $\text{Attn}_{\theta_{\text{Attn}}}(h) = (h_1^{\text{Attn}}, \dots, h_t^{\text{Attn}})$

3 Recovering the Weighted Majority Algorithm

We give a constructive implementation of the Weighted Majority algorithm (WMA) [Littlestone and Warmuth, 1994] using a constant-depth causal transformer equipped with continuous chain of thought via COCONUT. Our construction essentially maintains the state of WMA in a single context vector, which consists in a “superposition”, or linear combination, of the embeddings of the experts, where the weight of each expert’s embedding is precisely the logarithm of its weight. Under softmax attention, this readily allows predictions to be generated as the result of a context vector query against expert embedding keys. We use COCONUT in our construction to provide the continuous state token.

3.1 Problem Formulation

Problem setting. We consider the standard online full-information experts protocol with n experts which all make a binary prediction $p_{i,t} \in \{0, 1\}$ at each round. The learner sees these before making a prediction of her own, at which point the ground truth $y_t \in \{0, 1\}$ is revealed. The learner’s objective is to minimize the number of mistakes she makes, often compared against the most reliable expert.

The Weighted Majority Algorithm. The WMA maintains a positive weight for each expert $w_{i,t}$. The prediction of the algorithm at time t depends on the quantity

$$\hat{p}_t = \frac{\sum_{i: p_{i,t}=1} w_{i,t}}{\sum_i w_{i,t}} \quad (1)$$

In the randomized version of WMA, the algorithm predicts outcome 1 with probability $\hat{p}_t$; in the deterministic version, by binarizing at a suitable threshold (typically $1/2$). The multiplicative weights update method incorporates feedback: all experts who predicted correctly on that round have their weight increased by a factor of $\gamma > 1$. One may observe that $\log w_{i,t}$ is proportional to the number of times that expert i was correct up to time t ; this will be important to our implementation.

Token representations. In order to present it to the transformer, the algorithm’s input must be encoded into a sequence of tokens. We use a 4-block token embedding with positional extension, $d = 4d_{\text{TE}} + d_{\text{PE}}$ with $\mathbf{u}_v = [\text{id}(v) | \text{buf}_1(v) | \text{buf}_2(v) | \text{buf}_3(v) | \text{pos}(v)]^\top$, which contain the identity, buffer, and positional subspaces. The experts and binary possibilities are embedded via (arbitrary) orthonormal vectors in the identity subspace with empty buffer spaces. We write $\langle v \rangle$ to denote a token with identity embedding $\tilde{\mathbf{u}}_v$, where position is implicit. At each round, the input is the sequence of tokens

$$\underbrace{\langle w \rangle \langle z_t \rangle}_{\text{state slot}} \underbrace{\langle e_1 \rangle \langle p_{1,t} \rangle \dots \langle e_n \rangle \langle p_{n,t} \rangle}_{\text{advice dictionary}} \underbrace{\langle p? \rangle \langle y_t \rangle \langle w? \rangle}_{\text{query after prediction}}, \quad (2)$$

where $\langle y_t \rangle$ encodes the ground truth, and $\langle w \rangle, \langle p? \rangle, \langle w? \rangle$, are special tokens: $\langle w \rangle$ precedes the previous state, while $\langle p? \rangle$ and $\langle w? \rangle$ prompt the model for its prediction and updated state, respectively. For

the position embedding, we assume that there exists a matrix $\mathbf{P}$ on the positional embedding subspace such that $\mathbf{P}$ maps a positional embedding $\text{pos}(x)$ to the next positional embedding in sequence.

3.2 Transformer Construction for the Weighted Majority Algorithm

There are two tasks that the algorithm needs to perform: Given current expert weights and predictions, generate a prediction; then, once the ground truth is revealed, update the expert weights. The weights will be encoded in a superposition $\tilde{\mathbf{u}}_{z_t} = \sum_{i=1}^n \lambda_{i,t} \tilde{\mathbf{u}}_{e_i}$, where $e^{\lambda_{i,t}} = w_{i,t}$. Since the expert embeddings are orthogonal, $\tilde{\mathbf{u}}_{z_t} \cdot \tilde{\mathbf{u}}_{e_i} = \log w_{i,t}$. To generate a prediction, consider a soft attention head with keys $\tilde{\mathbf{u}}_{e_1}, \dots, \tilde{\mathbf{u}}_{e_n}$, values $\tilde{\mathbf{u}}_{p_{1,t}}, \dots, \tilde{\mathbf{u}}_{p_{n,t}} \in \{q_0, q_1\}$, and query $\tilde{\mathbf{u}}_{z_t}$. The attention head's output is

$$\hat{y}_t = \frac{\sum_{i=1}^n e^{\tilde{\mathbf{u}}_{z_t} \cdot \tilde{\mathbf{u}}_{e_i}} \tilde{\mathbf{u}}_{p_{i,t}}}{\sum_{i=1}^n e^{\tilde{\mathbf{u}}_{z_t} \cdot \tilde{\mathbf{u}}_{e_i}}} = \frac{\sum_{i=1}^n w_{i,t} \tilde{\mathbf{u}}_{p_{i,t}}}{\sum_{i=1}^n w_{i,t}} \Rightarrow \hat{y}_t \cdot q_1 = \frac{\sum_{i=1}^n w_{i,t} \mathbb{1}\{\tilde{\mathbf{u}}_{p_{i,t}} = q_1\}}{\sum_{i=1}^n w_{i,t}} = \frac{\sum_{i: p_{i,t}=1} w_{i,t}}{\sum_{i=1}^n w_{i,t}},$$

hao which is precisely the quantity upon which WMA's prediction is indexed. To update the weights, we have $\lambda_{i,t+1} = \lambda_{i,t} + \mathbb{1}\{p_{i,t} = y_t\} \cdot \log \gamma$, which is a linear operation on each component of the superposition. Therefore, we may rewrite

$$\tilde{\mathbf{u}}_{z_{t+1}} = \sum_{i=1}^n (\lambda_{i,t} + \eta \mathbb{1}\{y_t = p_{i,t}\}) \tilde{\mathbf{u}}_{e_i} = \sum_{i=1}^n \left[\begin{array}{c} \tilde{\mathbf{u}}_{z_t} \\ \log \gamma \cdot q_{p_{i,t}} \end{array} \right] \cdot \left[\begin{array}{c} \tilde{\mathbf{u}}_{z_t} \\ q_{y_t} \end{array} \right] \tilde{\mathbf{u}}_{e_i},$$

which is linear attention over appropriate keys and queries. In what follows, we will provide an implementation of the bookkeeping by which a transformer can extract the necessary information from the input sequence to perform these two operations, across three layers and five attention heads.

Theorem 3.1 (Exponential weights construction). *For n experts, horizon T , and learning rate η , there exists a causal transformer of $L = O(1)$ depth, $H = O(1)$ heads, and embedding dimension d , that outputs one continuous latent token, so that for every full-information expert sequence of length T , the model's prediction recovers exponential weights.*

3.2.1 Transformer Implementation

Each layer consists of one or more attention heads and a residual stream. The residual stream will propagate the layer input forward, while the output of each attention head is concatenated onto the forwarded input. In the interest of implementing Weighted Majority exactly, we freely intermix hard, softmax, and linear attention as necessary, but one could also use only the standard softmax attention and obtain a construction that implements an approximate version of the algorithm.

Layer 1, Attention Head 1: This head fetches the identity block of the next token in position, in particular, $\text{buf}_1(e_i)$ becomes $\tilde{\mathbf{u}}_{p_{i,t}}$. This can be accomplished with hard attention and query, key, and value matrices $\mathbf{Q}_{1,1} = [\mathbf{P} \ 0]$, $\mathbf{K}_{1,1} = [\mathbf{I} \ 0]$, and $\mathbf{V}_{1,1} = [0 \ \mathbf{I}]$.

Layer 2, Attention Head 1: When queried with identity blocks $\tilde{\mathbf{u}}_{p?}$ or $\tilde{\mathbf{u}}_{w?}$, this head outputs the superposition $\tilde{\mathbf{u}}_{z_t}$. This can be accomplished with hard attention and query, key, and value matrices

$$\mathbf{Q}_{2,1} = [\mathbf{I} \ 0 \ 0] \quad \mathbf{K}_{2,1} = [(\tilde{\mathbf{u}}_{p?} \otimes \tilde{\mathbf{u}}_w) + (\tilde{\mathbf{u}}_{w?} \otimes \tilde{\mathbf{u}}_{p?}) \ 0 \ 0] \quad \mathbf{V}_{2,1} = [0 \ \mathbf{I} \ 0].$$

Layer 2, Attention Head 2: When queried with identity block $\tilde{\mathbf{u}}_{w?}$, this head outputs the ground truth $\tilde{\mathbf{u}}_{y_t}$. The construction is similar to the above.

Layer 3, Attention Head 1: For the input $[\tilde{\mathbf{u}}_{p?} | \cdot | \tilde{\mathbf{u}}_{z_t} | \cdot | \text{pos}(p?)]$, arising from the previous layer, this head computes the weighted combination of expert predictions according to their current weights. This can be accomplished with soft attention and query, key, and value matrices

$$\mathbf{Q}_{3,1} = [0 \ 0 \ \mathbf{I} \ 0] \quad \mathbf{K}_{3,1} = [\mathbf{I} \ 0 \ 0 \ 0] \quad \mathbf{V}_{3,1} = [0 \ \mathbf{I} \ 0 \ 0].$$

Layer 3, Attention Head 2: For the input $[\tilde{\mathbf{u}}_{w?} | \cdot | \tilde{\mathbf{u}}_{z_t} | \tilde{\mathbf{u}}_{y_t} | \text{pos}(w?)]$, this head computes the updated superposition. This can be accomplished with linear attention and query, key, and value matrices

$$\mathbf{Q}_{3,2} = \begin{bmatrix} 0 & 0 & \mathbf{I} & 0 & 0 \\ 0 & 0 & 0 & \log \gamma \cdot \mathbf{I} & 0 \end{bmatrix} \quad \mathbf{K}_{3,2} = \begin{bmatrix} \mathbf{I} & 0 & 0 & 0 & 0 \\ 0 & \mathbf{I} & 0 & 0 & 0 \end{bmatrix} \quad \mathbf{V}_{3,2} = [\mathbf{I} \ 0 \ 0 \ 0 \ 0].$$

Layer 4: A final un-embedding layer maps the prediction vector to a token prediction. This is a linear feed-forward layer with one neuron per token, where each neuron's weights matches the identity embedding of its designated token, and the layer outputs a vector of token probabilities. Whether the transformer's output is random or thresholded (and whether it realizes randomized or hard weighted majority) depends on how these token probabilities are used to generate outputs.

4 Recovering Online Tabular Q -learning

In this section, we prove that a constant-depth transformer equipped with latent tokens can execute online Tabular Q -learning, a foundational off-policy reinforcement learning algorithm for solving Markov Decision Processes (MDPs) [Watkins and Dayan, 1992].

Problem setting. We model the environment as an infinite-horizon discounted MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma \rangle$, where $\mathcal{S}/\mathcal{A}$ are finite state/action spaces, $\mathcal{P}(s'|s, a)$ is the transition kernel, and $r(s, a) \in [0, 1]$ is a deterministic bounded reward. Let $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}_+$ denote the action-value function, with discount factor $\gamma \in (0, 1)$ and learning rate $\alpha \in (0, 1]$. The classical online tabular Q -learning algorithm maintains a tabular estimate Q_t and applies, at each step, the Bellman update

$$Q_{t+1}(s_t, a_t) \leftarrow Q_t(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a \in \mathcal{A}} Q_t(s_{t+1}, a) - Q_t(s_t, a_t) \right). \quad (3)$$

Tokens and embeddings. We specialize the general embedding framework above to the vocabulary $\mathcal{V} = \mathcal{S} \cup \mathcal{A} \cup \{ \langle \text{BOS} \rangle, \langle r \rangle, \langle \text{Select} \rangle, \langle Q_{\text{curr}} \rangle, \langle Q_{\text{next}} \rangle, \langle \text{Update} \rangle \}$, with orthonormal identity embeddings $\{ \tilde{\mathbf{u}}_v \}_{v \in \mathcal{V}} \subset \mathbb{R}^{d_{\text{TE}}}$. We use a 3-block token embedding with positional extension, $d = 3d_{\text{TE}} + d_{\text{PE}}$ with $\mathbf{u}_v = [\text{id}(v) | \text{buf}_1(v) | \text{buf}_2(v) | \text{pos}(v)]^\top$, which contain the identity, buffer, and positional subspaces.

Input structure. Each Q -learning step is encoded as a length- $(3|\mathcal{A}| + 9)$ sequence consisting of a context segment (encoding the current Q -table) followed by evaluation and update phases:

$$\langle \text{BOS} \rangle \ c_1, \dots, c_{|\mathcal{A}|} \| \langle Q_{\text{curr}} \rangle \langle s_t \rangle \langle a_t \rangle \langle r_t \rangle \langle Q_{\text{next}} \rangle \langle s_{t+1} \rangle (\langle s_{t+1} \rangle \langle a_i \rangle)_{i=1}^{|\mathcal{A}|} \langle \text{Select} \rangle \| \langle a^* \rangle \langle \text{Update} \rangle$$

Each context token c_a encodes the a -th column of Q_t via $\text{id}(c_a) = \tilde{\mathbf{u}}_a + \tilde{\mathbf{u}}_{\text{Update}}$, $\text{buf}_1(c_a) = \sum_{s \in \mathcal{S}} Q_t(s, a) \tilde{\mathbf{u}}_s$. The reward token satisfies $\text{id}(r_t) = \tilde{\mathbf{u}}_r$ and $\text{buf}_1(r_t) = r(s_t, a_t) \tilde{\mathbf{u}}_r$.

4.1 Transformer Construction for the Tabular Q -learning Algorithm

At each step, the model must perform two tasks: evaluate the current Q -values to select an action, and update the Q -table using the observed transition. Since all quantities are stored along orthogonal directions, the Bellman update (3) is implemented by writing each term into disjoint subspaces and summing them linearly in the residual stream. Here, we show how a small number of attention heads route and assemble these quantities to realize the update within a fixed-depth transformer.

Theorem 4.1 (Tabular Q -learning). *For any finite MDP with state space $\mathcal{S}$, action space $\mathcal{A}$, horizon T , learning rate $\alpha \in (0, 1]$, and discount $\gamma \in (0, 1)$, there exists a four-layer causal transformer TF_θ with $O(1)$ attention heads per layer, identity subspace dimension $d_{\text{TE}} = O(|\mathcal{S}| + |\mathcal{A}|)$, and a fixed parameter θ independent of T , such that running TF_θ autoregressively on the input structure above produces continuous thought tokens that implement the Q -learning update (3) at each step t .*

Proof sketch. We use a generalized fixed-offset attention mechanism inspired by Zhu et al. [2025c]: $\text{FO}(\mathcal{T}, -\ell)$ routes any token in a set $\mathcal{T}$ to a fixed position ℓ steps earlier. Under orthonormal embeddings, this acts as an exact indicator-based router and will be used to deterministically to focus attention from token-to-token. We provide a sketch below and defer the full proof to Section C.2.

Layer 1: Routing and context lookup. This layer stages all operands. State tokens are copied to the relevant positions, phase tags distinguish a_t from candidate actions, and each action retrieves its Q -table column. Head 1.1 copies the state tag to the action tokens by applying $\text{FO}(\mathcal{A}, -1)$ to copy the appropriate $\tilde{\mathbf{u}}_s$ into the first buffer of each action. Head 1.2 copies the state tag $\tilde{\mathbf{u}}_{s_{t+1}}$ to $\text{buf}_1(\text{Select})$ token’s buffer by applying $\text{FO}(\{\text{Select}\}, -2)$. Head 1.3 copies $\tilde{\mathbf{u}}_{s_t}$ into $\text{buf}_1(r_t)$. Head 1.4 uses a filtered fixed-offset selector to write $\tilde{\mathbf{u}}_{Q_{\text{curr}}}$ into $\text{id}(a_t)$. Head 1.5 uses causal content matching so that for each candidate, $\text{id}(a_i)$ ($i \geq 1$) receives $\tilde{\mathbf{u}}_{Q_{\text{next}}}$. Finally, in Head 1.6, each action token attends to its matching context token c_a and retrieves $\sum_s Q_t(s, a) \tilde{\mathbf{u}}_s$ into its buffer space.

Layer 2: Q -value evaluation. This layer computes scalar Q -values via inner products. In Head 2.1, $\langle \text{Select} \rangle$ attends to $\langle a_t \rangle$ and copies $\tilde{\mathbf{u}}_{s_t}$ into a buffer. In Head 2.2, a linear attention head computes $Q_t(s, a) = \tilde{\mathbf{u}}_s \cdot \sum_{s'} Q_t(s', a) \tilde{\mathbf{u}}_{s'}$ for each action token, and places the result into each action’s second buffer with $-Q_t(s_t, a_t) \tilde{\mathbf{u}}_{Q_{\text{curr}}}$ (current), $+Q_t(s_{t+1}, a) \tilde{\mathbf{u}}_{Q_{\text{next}}}$ (candidates).

Layer 3: Selection and maximization. This layer implements the max operator and selects actions. Head 3.1 uses softmax to treat $Q_t(s_{t+1}, a)$ as logits $\pi(a | s_{t+1}) \propto \exp(\beta Q_t(s_{t+1}, a))$. In

the limit $\beta \rightarrow \infty$, $\tilde{\mathbf{u}}_{a^*}$ adds into $\text{id}(\text{Select})$. Head 3.2 aggregates values $\sum_a \pi(a) Q_t(s_{t+1}, a) \rightarrow \max_a Q_t(s_{t+1}, a)$ within $\text{buf}_2(\text{Select})$. In Head 3.3, $\langle \text{Update} \rangle$ attends to $\langle a_t \rangle$ and retrieves $\tilde{\mathbf{u}}_{a_t}$ and $\sum_s Q_t(s, a_t) \tilde{\mathbf{u}}_s$.

Layer 4: Bellman update. This layer assembles the temporal-difference update in $\text{buf}_1(\text{Update})$. Head 4.1 subtracts the current value by extracting $-Q_t(s_t, a_t)$ and writing $-\alpha Q_t(s_t, a_t) \tilde{\mathbf{u}}_{s_t}$. Head 4.2 adds a reward by extracting r_t and writing $\alpha r_t \tilde{\mathbf{u}}_{s_t}$. Finally, Head 4.3 adds a discounted max by extracting $\max_a Q_t(s_{t+1}, a)$ and writing $\alpha \gamma \max_a Q_t(s_{t+1}, a) \tilde{\mathbf{u}}_{s_t}$.

By linearity of the residual stream, $\langle \text{Update} \rangle$ outputs 3, which is exactly $Q_{t+1}(s_t, a_t)$. The updated value is written back into the context representation, and autoregressively repeats the process.

Discussion 4.2 (Behavior policy). Using hard softmax ($\beta \rightarrow \infty$) in Head 3.1 recovers the greedy policy. Softening this recovers standard exploration policies: Boltzmann exploration sets an inverse temperature $\beta = 1/\tau$ for temperature τ and ϵ -greedy mixes the hard-softmax with uniform samples.

5 Experiments

Our theoretical results show that continuous latent contexts allow constant-depth transformers to implement online learning algorithms by storing and updating superposition states. We complement the theory with experiments to test whether such online algorithms can be learned by small transformers, whether the learned latent contexts encode the intended superposition states, and whether frontier LLMs are capable of making online decisions with or without a state-carrying reasoning context.

We conduct three sets of experiments¹. Section 5.1 studies expert prediction and shows that training a small transformer with latent contexts yields a small model that learns to make online decisions with similar performance as multiplicative weights, while its latent contexts recover expert-weight information. Section 5.2 moves beyond the expert setting to online tabular Q -learning, where the latent contexts need to store and update a richer algorithmic state, namely a Q -table. Finally, Section 5.3 evaluates frontier LLMs on interactive expert-prediction tasks under different prompt framings, and shows that allowing the model to output a short note carrying state information substantially improves its predictions.

5.1 Unsupervised Learning From Expert Data

We train a GPT-2 style decoder [Radford et al., 2019] Our dataset comprises of 3000 sequences, where each sequence length comprises of steps of length 100. A single continuous latent context vector is prepended to each step and updated autoregressively during the process. Next, the expert qualities are initiated uniformly at random between $[0.3, 0.9]$. At each step, there is a random binary true label, and each expert predicts the correct label with its probability (which is the quality), and gets a loss of 0 if correct and 1 if wrong. We train via a curriculum strategy where the losses are computed only at the masked positions, with loss $\mathcal{L} := \text{BCE}(\text{predicted decision}, \text{true label})$.

The curriculum strategy is that the model learns to reason over progressively larger MW sequences, one stage at a time. Specifically, for $1 \leq i \leq 10$, it trains on sequences truncated to $5i$ steps, and for $11 \leq i \leq 13$, it trains on sequences It trains on sequences truncated to $50 + 15(i - 10)$ steps. Every sequence is cut to STAGE steps so the model doesn't see longer reasoning chains than what the current stage allows. Also, with probability 0.1, shorter sequences from previous stages are mixed into the current stage's dataset, preventing catastrophic forgetting.

Attention behavior. Figure 2a shows that the model's performance on unsupervised learning from expert data is comparable to the optimal multiplicative weights algorithm with an inductive bias over the expert distributions. In Figure 2b, we see that the latent contexts M_t are significantly attended to by the other tokens, namely by the prediction tokens in the following step.

5.2 Imitation Learning of Tabular Q -learning

We train a pre-norm GPT-2 style decoder [Radford et al., 2019] (4 layers, 8 heads each, $d_{\text{model}} = 256$) with a recurrent continuous-context interface: a set of $|\mathcal{A}|$ learned context vectors is prepended to each step and updated autoregressively via the Update token, providing the only channel for cross-step

¹We provide our code in https://github.com/emiletimothy/transformer_decisionmaker.

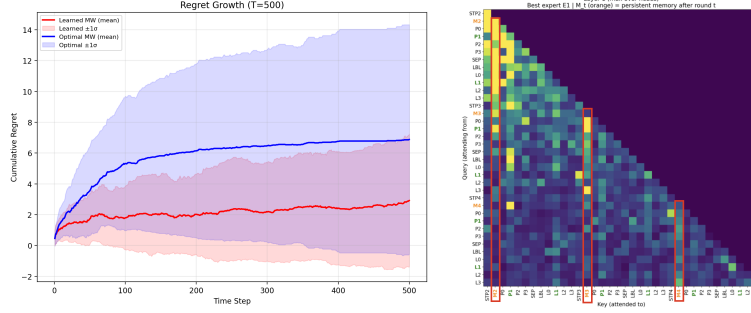


Figure 1: (Left) Cumulative regret plot, (Right) Causal attention heatmap on an MWU sequence

memory. Training data consists of 50,000 tabular Q -learning trajectories generated from randomly sampled finite MDPs, varying in state/action size, transition structure, reward distribution, and exploration rate. Each transition is labeled with the tabular greedy action $a^* = \arg \max_{a'} Q(s', a')$.

The Select token emits an action prediction $\hat{a}_t \sim \hat{p}_t$ at each step, and we minimize the per-step cross-entropy against the hindsight Q -learning greedy action $\mathcal{L} = \frac{1}{T} \sum_{t=1}^T \text{CE}(\hat{p}_t, a_t^*)$ with invalid actions masked when $|\mathcal{A}|$ varies. We employ a curriculum over action set size, gradually increasing $|\mathcal{A}|$ during training while grouping batches by action count to maintain a consistent context width. We evaluate the learned model along two axes: (i) performance relative to tabular Q -learning and (ii) evidence that the model internally implements the Q -learning update.

Behavioral equivalence. Figure 2 compares cumulative rewards of autonomous rollouts against tabular baselines. Across episodes, the transformer closely matches tabular Q -learning, indicating that it learns a policy of comparable quality under the same dynamics.

Evidence of Q -learning dynamics. We examine whether this behavioral similarity arises from an internal implementation of the Q -learning update. Figure 2 shows high per-step agreement with the tabular greedy action across timesteps, MDPs, and reward distributions, including under distribution shift, and that the Bellman update can be linearly decoded from the per-step context change, indicating that reward, current value, and bootstrapped value are explicitly represented in the residual stream.

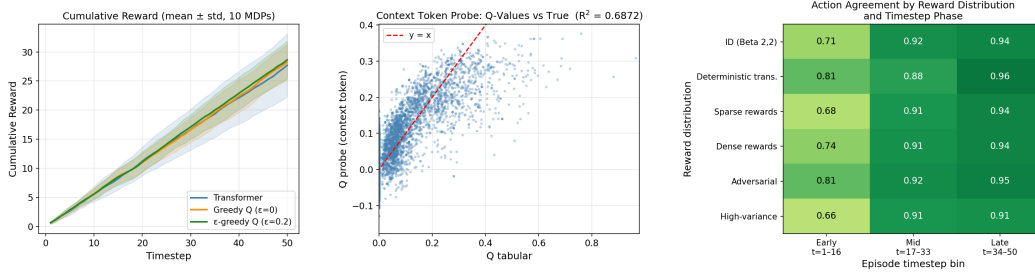


Figure 2: **Q -learning behavior and internal mechanism.** (Left) Learned transformer’s cumulative reward compared to tabular baselines. (Middle) A linear probe can recover the Bellman update signal from the context update, indicating explicit encoding of TD components. (Right) Per-step greedy-action agreement with tabular Q -learning across timesteps and reward distributions.

5.3 Inference on LLMs

Frontier LLMs are pretrained on a broad mixture of natural language, code, mathematical formulas, and algorithmic explanations. It is therefore natural to ask whether they have already internalized some form of online-learning behavior. In particular, when placed in a sequential prediction environment with feedback after each round, can such models improve their future predictions without any parameter updates? If so, what kind of online decision rule do they appear to use, and does their performance depend on having an explicit state-carrying context?

We study this question through an interactive expert-prediction inference task. The task is deliberately simple: there are four fixed experts, Expert A, Expert B, Expert C, and Expert D. At each round, each

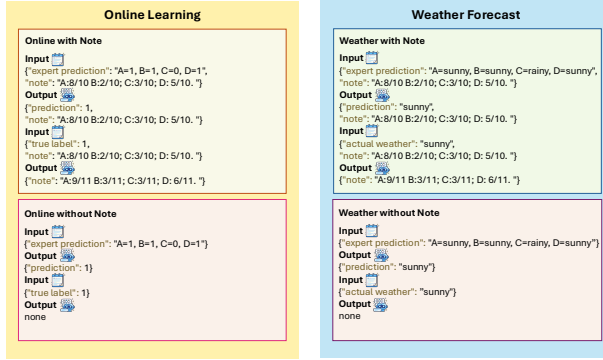


Figure 3: Prompt framing for LLM inference experiments. The note shown is an illustrative example that shows up in the experiments. The note is unsupervised.

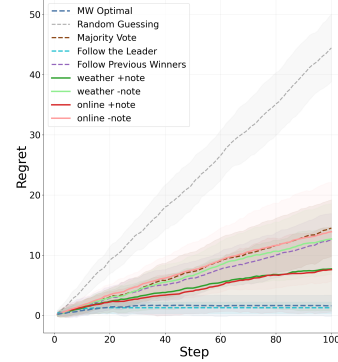


Figure 4: Cumulative regret of LLMs on interactive expert prediction against the best fixed expert on the same prefix.

expert makes a binary prediction, the LLM makes its own prediction based on these four experts, and then the true outcome is revealed to the LLM. The model is not told to run multiplicative weights or any other online learning algorithm explicitly. Its objective is to be as accurate as possible over time.

Prompt framing. As shown in Figure 3, we evaluate four prompting schemes obtained by crossing two task framings with two state-carrying protocols. The *online* framing presents the task as binary expert prediction with labels 0 and 1, while the *weather* framing presents the same sequences as weather forecasts with labels “sunny” and “rainy”, using the mapping $1 \mapsto \text{“sunny”}$ and $0 \mapsto \text{“rainy”}$. The two framings differ only in surface language.

For each framing, we compare a *no-note* protocol and a *note* protocol. In the no-note protocol, the model outputs a prediction, the ground truth is appended to the interaction history as feedback, and the next round begins. In the note protocol, the model is allowed to output a short note after observing the feedback. This note is passed back to the model in the next round and can contain any information the model considers useful, such as which experts appear reliable. The note is unsupervised and is not forced to follow any particular format, serving as a controlled, visible state channel.

Evaluation. We evaluate each protocol by cumulative regret against the best fixed expert on each prefix of the sequence. Figure 4 reports the cumulative regret for DeepSeek-V3. The plot shows that DeepSeek-V3 exhibits some online adaptation from feedback, but its performance depends strongly on the availability of an explicit note. Without a note, the model’s curves are close to MAJORITY VOTE and FOLLOW PREVIOUS WINNER baselines, suggesting that it often relies on short-term feedback rather than maintaining a stable estimate of expert reliability. Allowing the model to write a short note carrying expert-reliability information substantially improves prediction performance and clearly outperforms this short-memory baseline, suggesting that a visible state-carrying context helps LLMs use feedback over long interactions. However, the most successful natural language notes have size linear in the number of experts, in contrast to our construction, which uses a constant number of continuous latent context tokens. Detailed setup, full prompts, implementation details, Qwen-3-14B results, additional ablations, and note analyses are provided in Section E.3.

6 Conclusion

We explore how transformers can effectively make online decisions, using small continuous latent contexts to maintain a summary of history and encode an algorithmic state. We study two problems – learning from experts, and tabular RL – and give theoretical constructions to implement classical algorithms for these problems (Weighted Majority and Q -learning) using the latent context to maintain the algorithmic state. We show that gradient-based training on a prediction-based (experts) and imitation-based (RL) loss learns to use the context effectively, hinting that the gradient-based solution may be similar to the target algorithms. Finally, we study the performance of two open-source LLMs on the experts problem in various settings, ablating on the ability to maintain a contextual “note”, and show that they fall short of the Weighted Majority baseline, indicating that small continuous latent contexts may provide practical advantages for online decision-making in LLMs.

Acknowledgements

We thank Sarah Liaw, Mirabel Reid, and Jacob Abernethy for insightful discussions. EA’s research is supported by NSF Grant CCF 2338816, XC’s research is supported by NSF award CCF-2106444 and a JPMC AI PhD fellowship, and MD’s research is supported by an NSF ARNI postdoctoral research fellowship.

Limitations and Future Work

We identify several directions of future work. Future theoretical work should determine whether these implementations can emerge from gradient-based training. Since those algorithms are worst-case, optimal distributional information may allow modified algorithms to perform substantially better, so this question is likely to be deep. A related question with both theoretical and empirical angles is the circumstances under which various RL approaches are optimal, with well-known statistical/computational tradeoffs for model-based versus model-free approaches. Here we only considered imitation learning of an observed Q -learner; a substantial extension of this work would be to identify the emergence of Q -learning when training to maximize reward. Finally, we restricted our study to simplified decision-making settings, and it is unclear to what extent our results may generalize to real-world reasoning.

Impact Statement

There are substantial risks involved in creating decision-making models with reasoning steps which are not human interpretable. A great deal of the confidence we have in the alignment of large agentic systems is because we can directly monitor their reasoning, so the profusion of frontier models with more opaque reasoning could present a substantial challenge to the already difficult fields of interpretability and safety. On the positive side, our results contribute to a body of work suggesting that continuous latent reasoning may offer significant efficiency gains, with the potential to mitigate AI’s consumption of scarce resources.

References

- Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. What learning algorithm is in-context learning? Investigations with linear models, 2023.
- Ido Amos, Avi Caciularu, Mor Geva, Amir Globerson, Jonathan Herzig, Lior Shani, and Idan Szpektor. Latent Reasoning with Supervised Thinking States. *arXiv preprint arXiv:2602.08332*, 2026. URL <https://arxiv.org/abs/2602.08332>.
- Emile Anand and Ishani Karmarkar. Learning Approximate Nash Equilibria in Cooperative Multi-Agent Reinforcement Learning via Mean-Field Subsampling, 2026.
- Emile Anand and Guannan Qu. Efficient Reinforcement Learning for Global Decision Making in the Presence of Local Agents at Scale, 2024.
- Emile Timothy Anand and Sarah Liaw. Feel-Good Thompson Sampling for Contextual Bandits: a Markov Chain Monte Carlo Showdown. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025.
- Emile Timothy Anand, Ishani Karmarkar, and Guannan Qu. Mean-Field Sampling for Cooperative Multi-Agent Reinforcement Learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Yi Cao, Weijie Shi, Wei-Jie Xu, Yucheng Shen, Yue Cui, Hanghui Guo, Shimin Di, Ziyi Liu, Jiaming Li, Alexander Zhou, Jia Zhu, and Jiajie Xu. TIV: Thought Injection via Vectors for Efficient Reasoning in Large Reasoning Models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(36):30148–30155, Mar. 2026. doi: 10.1609/aaai.v40i36.40264.
- Bryan Chan, Xinyi Chen, András György, and Dale Schuurmans. Toward Understanding In-context vs. In-weight Learning, 2025. URL <https://arxiv.org/abs/2410.23042>.

- Lijie Chen, Binghui Peng, and Hongxun Wu. Theoretical limitations of multi-layer Transformer, 2024. URL <https://arxiv.org/abs/2412.02975>.
- David Chiang, Peter Cholak, and Anand Pillay. Tighter bounds on the expressivity of transformer encoders. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 5544–5562. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/chiang23a.html>.
- Alon Cohen and Tamir Hazan. Following the perturbed leader for online structured learning. In *International Conference on Machine Learning*, pages 1034–1042. PMLR, 2015.
- DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanbiao Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. DeepSeek-V3 Technical Report, 2025. URL <https://arxiv.org/abs/2412.19437>.
- Hanwen Du, Yuxin Dong, and Xia Ning. Latent Thinking Optimization: Your Latent Reasoning Language Model Secretly Encodes Reward Signals in Its Latent Thoughts, 2026.
- Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. Towards revealing the mystery behind chain of thought: a theoretical perspective. *Advances in Neural Information Processing Systems*, 36:70757–70798, 2023.
- Abraham D. Flaxman, Adam Tauman Kalai, and H. Brendan McMahan. Online convex optimization in the bandit setting: gradient descent without a gradient, 2004.
- Deqing Fu, Tianqi CHEN, Robin Jia, and Vatsal Sharan. Transformers Learn Higher-Order Optimization Methods for In-Context Learning: A Study with Linear Models, 2024.
- Khashayar Gatmiry, Nikunj Saunshi, Sashank J. Reddi, Stefanie Jegelka, and Sanjiv Kumar. Can Looped Transformers Learn to Implement Multi-step Gradient Descent for In-context Learning?, 2024. URL <https://arxiv.org/abs/2410.08292>.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R Bartoldson, Bhavya Kailkhura, Abhinav Bhatle, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025.

- Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training Language Models With Pause Tokens, 2024.
- Wes Gurnee and Max Tegmark. Language models represent space and time. *arXiv preprint arXiv:2310.02207*, 2023. URL <https://arxiv.org/abs/2310.02207>.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training Large Language Model to Reason in a Continuous Latent Space, 2025.
- Elad Hazan. Introduction to Online Convex Optimization, 2023.
- Elad Hazan, Karan Singh, and Cyril Zhang. Learning Linear Dynamical Systems via Spectral Filtering, 2017. URL <https://arxiv.org/abs/1711.00946>.
- Jianhao Huang, Zixuan Wang, and Jason D Lee. Transformers learn to implement multi-step gradient descent with chain of thought. *arXiv preprint arXiv:2502.21212*, 2025.
- Francesco Innocenti and El Mehdi Achour. A Simple Generalisation of the Implicit Dynamics of In-Context Learning, 2025. URL <https://arxiv.org/abs/2512.11255>.
- Adam Kalai and Santosh Vempala. Efficient algorithms for online decision problems. *Journal of Computer and System Sciences*, 71(3):291–307, 2005.
- Jungtaek Kim, Thomas Zeng, Ziqian Lin, Minjae Lee, Chungpa Lee, Jy yong Sohn, Hyung Il Koo, and Kangwook Lee. Transformers in the Dark: Navigating Unknown Search Spaces via Bandit Feedback. *Transactions on Machine Learning Research*, 2026. ISSN 2835-8856. URL <https://openreview.net/forum?id=Jij7zCjVfc>. J2C Certification.
- Deqian Kong, Dehong Xu, Minglu Zhao, Bo Pang, Jianwen Xie, Andrew Lizarraaga, Yuhao Huang, Sirui Xie, and Ying Nian Wu. Latent plan transformer for trajectory abstraction: Planning as latent space inference. *Advances in Neural Information Processing Systems*, 37:123379–123401, 2024.
- Michael Laskin, Luyu Wang, Junhyuk Oh, Emilio Parisotto, Stephen Spencer, Richie Steigerwald, DJ Strouse, Steven Hansen, Angelos Filos, Ethan Brooks, et al. In-context reinforcement learning with algorithm distillation. *arXiv preprint arXiv:2210.14215*, 2022.
- Hongkang Li, Songtao Lu, Pin-Yu Chen, Xiaodong Cui, and Meng Wang. Training Nonlinear Transformers for Chain-of-Thought Inference: A Theoretical Generalization Analysis. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. Chain of Thought Empowers Transformers to Solve Inherently Serial Problems, 2024. URL <https://arxiv.org/abs/2402.12875>.
- Yiheng Lin, James A. Preiss, Emile Anand, Yingying Li, Yisong Yue, and Adam Wierman. Online adaptive policy selection in time-varying systems: No-regret via contractive perturbations. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 53508–53521. Curran Associates, Inc., 2023.
- Yiheng Lin, James A. Preiss, Fengze Xie, Emile Anand, Soon-Jo Chung, Yisong Yue, and Adam Wierman. Online policy optimization in unknown nonlinear systems. In Shipra Agrawal and Aaron Roth, editors, *Proceedings of Thirty Seventh Conference on Learning Theory*, volume 247 of *Proceedings of Machine Learning Research*, pages 3475–3522. PMLR, 30 Jun–03 Jul 2024a.
- Ying-Chun Lin, Jennifer Neville, Jack Stokes, Longqi Yang, Tara Safavi, Mengting Wan, Scott Counts, Siddharth Suri, Reid Andersen, Xiaofeng Xu, Deepak Gupta, Sujay Kumar Jauhar, Xia Song, Georg Buscher, Saurabh Tiwary, Brent Hecht, and Jaime Teevan. Interpretable user satisfaction estimation for conversational systems with large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11100–11115, Bangkok, Thailand, August 2024b. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.598.
- Nick Littlestone and Manfred K Warmuth. The weighted majority algorithm. *Information and Computation*, 108(2):212–261, 1994.

- Bo Liu, Rui Wang, Lemeng Wu, Yihao Feng, Peter Stone, and Qiang Liu. Longhorn: State space models are amortized online learners. *arXiv preprint arXiv:2407.14207*, 2024.
- William Merrill and Ashish Sabharwal. The Expressive Power of Transformers with Chain of Thought, 2024. URL <https://arxiv.org/abs/2310.07923>.
- William Merrill and Ashish Sabharwal. A Little Depth Goes a Long Way: The Expressive Power of Log-Depth Transformers, 2025. URL <https://arxiv.org/abs/2503.03961>.
- Jacob Pfau, William Merrill, and Samuel R. Bowman. Let’s Think Dot by Dot: Hidden Computation in Transformer Language Models, 2024. URL <https://arxiv.org/abs/2404.15758>.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Keshav Ramji, Tahira Naseem, and Ramón Fernandez Astudillo. Thinking Without Words: Efficient Latent Reasoning with Abstract Chain-of-Thought, 2026.
- Vedant Shah, Johan Obando-Ceron, Vineet Jain, Brian Bartoldson, Bhavya Kailkhura, Sarthak Mittal, Glen Berseth, Pablo Samuel Castro, Yoshua Bengio, Nikolay Malkin, Moksh Jain, Siddarth Venkatraman, and Aaron Courville. A Comedy of Estimators: On KL Regularization in RL Training of LLMs, 2026. URL <https://arxiv.org/abs/2512.21852>.
- Max Simchowitz, Karan Singh, and Elad Hazan. Improper learning for non-stochastic control. In Jacob Abernethy and Shivani Agarwal, editors, *Proceedings of Thirty Third Conference on Learning Theory*, volume 125 of *Proceedings of Machine Learning Research*, pages 3320–3436. PMLR, 09–12 Jul 2020. URL <https://proceedings.mlr.press/v125/simchowitz20a.html>.
- Lena Strobl, William Merrill, Gail Weiss, David Chiang, and Dana Angluin. What Formal Languages Can Transformers Express? A Survey. *Transactions of the Association for Computational Linguistics*, 12:543–561, 2024. ISSN 2307-387X. doi: 10.1162/tacl_a_00663.
- Eric Todd, Millicent Li, Arnab Sen Sharma, Aaron Mueller, Byron C Wallace, and David Bau. Function vectors in large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=AwxytyMwaG>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 5998–6008, 2017. URL <https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>.
- Archer Wang, Emile Anand, Yilun Du, and Marin Soljačić. Unsupervised Decomposition and Recombination with Discriminator-Driven Diffusion Models, 2026.
- Jiuqi Wang, Ethan Blaser, Hadi Daneshmand, and Shangdong Zhang. Transformers can learn temporal difference methods for in-context reinforcement learning. *arXiv preprint arXiv:2405.13861*, 2024a.
- Xinyi Wang, Lucas Caccia, Oleksiy Ostapenko, Xingdi Yuan, William Yang Wang, and Alessandro Sordani. Guiding Language Model Reasoning with Planning Tokens, 2024b.
- Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3):279–292, 1992. doi: 10.1007/BF00992698. URL <https://doi.org/10.1007/BF00992698>.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, 2023. URL <https://arxiv.org/abs/2201.11903>.
- Jianzhe Wei, Siyu Chen, Jianliang He, and Zhuoran Yang. HOW TRANSFORMERS LEARN CAUSAL STRUCTURES IN-CONTEXT: EXPLAINABLE MECHANISM MEETS THEORETICAL GUARANTEE. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=bpF8zgSt41>.
- Tengyang Xie, Dylan J. Foster, Akshay Krishnamurthy, Corby Rosset, Ahmed Awadallah, and Alexander Rakhlin. Exploratory Preference Optimization: Harnessing Implicit Q*-Approximation for Sample-Efficient RLHF, 2024. URL <https://arxiv.org/abs/2405.21046>.

- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.
- Kunhe Yang, Lin F. Yang, and Simon S. Du. *Q*-learning with Logarithmic Regret, 2021.
- Hanlin Zhu, Shibo Hao, Zhiting Hu, Jiantao Jiao, Stuart Russell, and Yuandong Tian. Emergence of superposition: Unveiling the training dynamics of chain of continuous thought. *arXiv preprint arXiv:2509.23365*, 2025a.
- Hanlin Zhu, Shibo Hao, Zhiting Hu, Jiantao Jiao, Stuart Russell, and Yuandong Tian. Reasoning by superposition: A theoretical perspective on chain of continuous thought. *arXiv preprint arXiv:2505.12514*, 2025b.
- Hanlin Zhu, Shibo Hao, Zhiting Hu, Jiantao Jiao, Stuart Russell, and Yuandong Tian. Reasoning by Superposition: A Theoretical Perspective on Chain of Continuous Thought, 2025c.
- Martin Zinkevich. Online convex programming and generalized infinitesimal gradient ascent. In *Proceedings of the 20th international conference on machine learning (icml-03)*, pages 928–936, 2003.

Outline of the Appendix

- Section A: Table of notation for our theoretical results.
- Section B: Theoretical results showing the optimality of the Weighted Majority Algorithm under a particular loss.
- Section C: Full Tabular Q -learning construction: Generalized Attention Chooser and main proof
- Section D: Empirical validation of our theoretical constructions by implementations.
- Section E: Details to our experiments.

A Mathematical Background

Basic notations. For any integer $N > 0$, we use $[N]$ to denote the set $\{1, 2, \dots, N\}$. For any finite set $\mathcal{X}$, let $|\mathcal{X}|$ denote the cardinality of $\mathcal{X}$. Let $\mathbb{R}$ be the set of real numbers. We use $\mathbb{1}\{\cdot\}$ to denote the indicator function. We use lower-case bold letters (e.g., $\mathbf{x}, \theta$) and upper-case bold letters (e.g., $\mathbf{W}, \mathbf{U}$) to denote vectors and matrices, respectively. In particular, we use $\mathbf{I}_d$ to denote a $d \times d$ identity matrix, use $\mathbf{0}_{m \times n}$ (or $\mathbf{0}_m$) to denote an $m \times n$ zero matrix (or an m -dimensional zero vector), and use $\mathbf{e}_i$ to denote a one-hot vector of which the i -th entry is one and other entries are all zero, where the dimension of $\mathbf{e}_i$ can be inferred from the context. We also use $\|\cdot\|_\infty$ and $\|\cdot\|_2$ to represent ℓ_∞ and ℓ_2 norm, respectively. For vectors $\mathbf{u} \in \mathbb{R}^m$ and $\mathbf{v} \in \mathbb{R}^n$, let $\mathbf{u} \otimes \mathbf{v} = \mathbf{u}\mathbf{v}^\top \in \mathbb{R}^{m \times n}$ denote their inner product. Finally, for any vector $\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^d$, we define the softmax function $\text{SoftMax} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ as $\text{SoftMax}(\mathbf{x})_i = \exp(x_i) / (\sum_{j=1}^d \exp(x_j))$.

Tokens and Embeddings. Fix a vocabulary $\mathcal{V} = [V]$. Each token $v \in \mathcal{V}$ has an embedding $\mathbf{u}_v \in \mathbb{R}^d$, whose dimension $d = k d_{\text{TE}} + d_{\text{PE}}$ decomposes as k token-embedding blocks and one positional block. For a vector $\mathbf{x} = (x_1, x_2, \dots, x_d)^\top \in \mathbb{R}^d$, we define $\text{id}(\mathbf{x}) = (x_1, \dots, x_{d_{\text{TE}}})^\top$, $\text{buf}_j(\mathbf{x}) = (x_{j \cdot d_{\text{TE}} + 1}, \dots, x_{(j+1) \cdot d_{\text{TE}}})^\top$ for $j \in [k]$, and $\text{pos}(\mathbf{x}) = (x_{k \cdot d_{\text{TE}} + 1}, \dots, x_d)$. Let $\tilde{\mathbf{u}}_v = \text{id}(\mathbf{u}_v) \in \mathbb{R}^{d_{\text{TE}}}$ for $v \in \mathcal{V}$. Next, let $\mathbf{U} = [\tilde{\mathbf{u}}_1, \tilde{\mathbf{u}}_2, \dots, \tilde{\mathbf{u}}_V] \in \mathbb{R}^{d_{\text{TE}} \times V}$ and assume token orthonormality, $\mathbf{U}^\top \mathbf{U} = \mathbf{I}_V$.

A.1 Token Vocabulary

A.1.1 Multiplicative Weights Setting

A shared discrete vocabulary of size $n + 5$ is used, where n is the number of experts:

Token(s)	Meaning
$\langle w \rangle$	state slot marker (precedes weight superposition)
$\langle z_t \rangle$	weight superposition $\tilde{\mathbf{u}}_{z_t} = \sum_{i=1}^n \lambda_{i,t} \tilde{\mathbf{u}}_{e_i}$
$\langle e_1 \rangle, \dots, \langle e_n \rangle$	expert tokens
$\langle p_{1,t} \rangle, \dots, \langle p_{n,t} \rangle$	expert prediction tokens, valued in $\{q_0, q_1\}$
$\langle q_0 \rangle, \langle q_1 \rangle$	binary outcome tokens
$\langle y_t \rangle$	ground-truth label token
$\langle p? \rangle$	prediction query token
$\langle w? \rangle$	weight-update query token

A.1.2 Tabular Q -learning Setting

A shared discrete vocabulary of size $2 + S_{\max} + A_{\max} + 5$ is used across all episodes:

Token(s)	Meaning
$\langle \text{BOS} \rangle$	sequence start
$\langle s_1 \rangle, \dots, \langle s_{S_{\max}} \rangle$	state tokens
$\langle a_1 \rangle, \dots, \langle a_{A_{\max}} \rangle$	action tokens
$\langle r \rangle$	reward marker
$\langle \text{Select} \rangle$	greedy-action query token
$\langle Q_{\text{curr}} \rangle, \langle Q_{\text{next}} \rangle, \langle \text{Update} \rangle$	Q -learning phase markers

State and action tokens are pre-allocated up to $S_{\max}$ and $A_{\max}$ so that MDPs of varying sizes share a unified vocabulary.

B Population log-loss targets the MWU predictor

In this section, we show that under a latent-expert data-generating model, the population log-loss minimizer is the Bayesian posterior predictive mixture, whose weights follow multiplicative weights.

B.1 Proof that under Binary Cross-Entropy, dynamics converges to MWU

The setup is that $\mathcal{H}_t$ denotes the history available at round t , which is the current expert advice and the past outcomes. Let expert i emit a predictive distribution $p_{i,t}(\cdot) \in \Delta_K$. Let the latent state token encode logits $\lambda_t \in \mathbb{R}^n$, and the normalized expert weights are $w_{t,i} = \text{softmax}(\lambda_t)_i$, and the model is trained by sequence log-loss

$$\mathcal{L}(\theta) = \mathbb{E} \left[\sum_{t=1}^T -\log q_{\theta,t}(Y_t | \mathcal{H}_t) \right]. \quad (4)$$

We assume that the data is generated by a latent expert $I^* \sim \pi$ and $Y_t | (I^* = i, \mathcal{H}_t) \sim p_{i,t}(\cdot)$.

Lemma B.1 (Log-loss is uniquely minimized the true conditional). *For a true distribution P and a predicted distribution Q , we have that $\mathbb{E}_{Y \sim P}[-\log Q(Y)] = H(P) + \text{KL}(P \| Q)$.*

Proof. By expanding the LHS, we have

$$\begin{aligned} \mathbb{E}_{Y \sim P}[-\log Q(Y)] &= - \sum_y P(y) \log Q(y) \\ &= - \sum_y P(y) \log P(y) + \sum_y P(y) \log \frac{P(y)}{Q(y)} \\ &= H(P) + \text{KL}(P \| Q). \end{aligned}$$

Since the KL divergence is non-negative and 0 if and only if $P = Q$, the minimizer is unique. $\square$

Theorem B.2. *Define $q_t^*(\cdot | \mathcal{H}_t) := P(Y_t = \cdot | \mathcal{H}_t)$. Let q^* be the minimizer of $\mathcal{L}$ and it has the form $q_t^*(\cdot | \mathcal{H}_t) = \sum_{i=1}^n (\cdot | \mathcal{H}_t) = \sum_{i=1}^n w_{t,i} p_{i,t}(\cdot)$ and $w_{t,i} = P(I^* = i | \mathcal{H}_t)$. Then, the posterior weights satisfy $w_{t+1,i} = \frac{w_{t,i} p_{i,t}(Y_t)}{\sum_{j=1}^n w_{t,j} p_{j,t}(Y_t)}$. Moreover, with per-expert log-loss $\ell_{i,t} = -\log p_{i,t}(Y_t)$, this update recovers the multiplicative weights update.*

Proof. By the tower property, we have that

$$\mathcal{L}(\theta) = \sum_{t=1}^T \mathbb{E}[\mathbb{E}[-\log q_{\theta,t}(Y_t | \mathcal{H}_t)] | \mathcal{H}_t],$$

where for any fixed history h , Theorem B.1 shows that $\mathbb{E}[-\log q_{\theta,t}(Y_t | \mathcal{H}_t)]$ is uniquely minimized by

$$q_{\theta,t}(\cdot | h) = \Pr[Y_t = \cdot | \mathcal{H}_t = h],$$

and hence the unique population minimizer is q^* . Now, we compute the conditional distribution under the latent expert model

$$\Pr[Y_t = y | \mathcal{H}_t] = \sum_{i=1}^m \Pr[I^* = i | \mathcal{H}_t] \Pr[Y_t = y | I^* = i, \mathcal{H}_t] = \sum_{i=1}^n w_{t,i} p_{i,t}(y),$$

and hence q_t^* is the expert mixture. Next, after observing Y_t , Bayes' rule gives that

$$w_{t+1,i} = \Pr[I^* = i | \mathcal{H}_t, Y_t] = \frac{\Pr[Y_t | I^* = i, \mathcal{H}_t] \Pr[I^* = i | \mathcal{H}_t]}{\sum_j \Pr[Y_t | I^* = j, \mathcal{H}_t] \Pr[I^* = j | \mathcal{H}_t]}.$$

Then,

$$\Pr[Y_t | I^* = i, \mathcal{H}_t] = p_{i,t}(Y_t) \implies w_{t+1,i} = \frac{w_{t,i} p_{i,t}(Y_t)}{\sum_{j=1}^n w_{t,j} p_{j,t}(Y_t)}.$$

Finally, writing $p_{i,t}(Y_t) = e^{-\ell_{i,t}}$ recovers the multiplicative weights update. $\square$

Corollary B.3. *Suppose that the transformer with latent context contains the multiplicative weights update predictor. Then, any global minimizer of the population log-loss implements the MWU predictor exactly, and if gradient-based training returns $\hat{\theta}$ with excess population risk at most ϵ such that $\mathcal{L}(\hat{\theta}) - \mathcal{L}(\theta^*) \leq \epsilon$, then*

$$\sum_{t=1}^T \mathbb{E}[\text{KL}(q_t^*(\cdot | \mathcal{H}_t) \| q_{\hat{\theta},t}(\cdot | \mathcal{H}_t))] \leq \epsilon. \quad (5)$$

Proof. Since $q_t^*(\cdot | \mathcal{H}_t) = \Pr[Y_t = \cdot | \mathcal{H}_t]$, applying Theorem B.1 gives that

$$-\mathbb{E}[\log q_{\hat{\theta},t}(Y_t | \mathcal{H}_t) | \mathcal{H}_t] - \mathbb{E}[-\log q_t^*(Y_t | \mathcal{H}_t) | \mathcal{H}_t] = \text{KL}(q_t^*(\cdot | \mathcal{H}_t) \| q_{\hat{\theta},t}(\cdot | \mathcal{H}_t)).$$

Therefore, by the tower property,

$$\sum_{t=1}^T \mathbb{E}[\text{KL}(q_t^*(\cdot | \mathcal{H}_t) \| q_{\hat{\theta},t}(\cdot | \mathcal{H}_t))] = \mathcal{L}(\hat{\theta}) - \mathcal{L}(\theta^*) \leq \epsilon,$$

which proves the corollary. $\square$

Moreover, the gradient field of the one-step log-loss is the displacement from the current expert mixture to the multiplicative weights posterior. We formalize this below.

Lemma B.4. *Fix one round and define $r_{i,t} := p_{i,t}(Y_t)$ and $L_t(\lambda) = -\log \sum_{i=1}^n \text{softmax}(\lambda)_i r_{i,t}$. Let $w = \text{softmax}(\lambda)$, and let $w_i^+ = \frac{w_i r_{i,t}}{\sum_j w_j r_{j,t}}$. Then, w^+ is the multiplicative weights update and*

$$\nabla_{\lambda} L_t(\lambda) = w - w^+. \quad (6)$$

Proof. Let $q(\lambda) = \sum_j w_j r_{j,t}$. For the softmax function, we have that

$$\begin{aligned} \frac{\partial L_t}{\partial \lambda_i} &= -\frac{1}{q} \frac{\partial q}{\partial \lambda_i} \\ &= -\frac{1}{q} \sum_j r_{j,t} \frac{\partial w_j}{\partial \lambda_i} \\ &= -\frac{1}{q} \sum_j r_{j,t} w_j (\mathbb{1}\{i = j\} - w_i) \\ &= -\frac{1}{q} w_i (r_{i,t} - q) = w_i - \frac{w_i r_{i,t}}{q} = w_i - w_i^+, \end{aligned}$$

which completes the proof. $\square$

C Proofs for the Tabular Q-learning Construction

C.1 Proof of the Generalized Attention Chooser

In this section, we provide the formal construction and proof for the generalized attention chooser $\text{FO}(\mathcal{T}, -\ell)$.

Lemma C.1 (Generalized Attention Chooser). *Fix any token subset $\mathcal{T} \subseteq \mathcal{V}$, shift $\ell \geq 0$, and error tolerance $\varepsilon \in (0, 1)$. Under sinusoidal positional encoding and orthogonal token embeddings, there exists a construction of query and key matrices $Q, K \in \mathbb{R}^{(2d_{PE}) \times d}$ such that the resulting attention head $\text{FO}(\mathcal{T}, -\ell)$ satisfies:*

For any sequence $\mathbf{h}_{[T]}$ and any position $i \in [T]$:

- If $\langle \tilde{\mathbf{h}}_i, \tilde{\mathbf{u}}_v \rangle = 1$ for some $v \in \mathcal{T}$ (i.e., the token is in $\mathcal{T}$), then the attention score to $i - \ell$ satisfies $s_{i,i-\ell} \geq 1 - \varepsilon$.
- Otherwise, the attention score to the BOS sink satisfies $s_{i,1} \geq 1 - \varepsilon$.

where $s_{i,j}$ is the softmax attention score from position i to j .

Proof. Following the notation of Zhu et al. [2025c], for each position i in the token sequence, there is a corresponding positional encoding $\mathbf{p}_i = \text{PosEncode}_{\theta_{PE}}(i) \in \mathbb{R}^d$, where only the last d_{PE} entries are non-zero. Let $\bar{\mathbf{p}} = \text{pos}(\mathbf{p}_i)$. Additionally, let

$$\tilde{\mathbf{u}}_{\bar{\mathcal{T}}} = \sum_{v \notin \mathcal{T}} \tilde{\mathbf{u}}_v$$

denote the unnormalized superposition of all token embeddings outside $\mathcal{T}$.

We assume the representation is a direct sum $\mathcal{H} = \mathcal{H}_{\text{token}} \oplus \mathcal{H}_{\text{pos}}$ with $\mathcal{H}_{\text{token}} \perp \mathcal{H}_{\text{pos}}$. The vocabulary is mapped to a $3d_{TE}$ dimensional buffer and concatenated with the positional encodings of dimension d_{PE} , yielding $d = 3d_{TE} + d_{PE}$.

We construct the query and key projection matrices $Q, K \in \mathbb{R}^{(2d_{PE}) \times d}$ in block form. Let $\otimes$ denote the outer product:

$$Q = \begin{bmatrix} 0_{d_{PE} \times d_{TE}} & 0_{d_{PE} \times d_{TE}} & 0_{d_{PE} \times d_{TE}} & I_{d_{PE}} \\ \xi \bar{\mathbf{p}}_1 \otimes \tilde{\mathbf{u}}_{\bar{\mathcal{T}}} & 0_{d_{PE} \times d_{TE}} & 0_{d_{PE} \times d_{TE}} & 0_{d_{PE} \times d_{TE}} \end{bmatrix}, \quad K = \begin{bmatrix} 0_{d_{PE} \times 3d_{TE}} & \eta R^{(\ell)} \\ 0_{d_{PE} \times 3d_{TE}} & \eta I_{d_{PE}} \end{bmatrix},$$

where $\xi, \eta > 0$ are scaling constants to be specified, and $R^{(\ell)}$ is the shift matrix for sinusoidal positional encodings (such that $R^{(\ell)} \bar{\mathbf{p}}_i = \bar{\mathbf{p}}_{i+\ell}$).

For any position i and j , evaluating $Q(\mathbf{h}_i + \mathbf{p}_i)$ and $K(\mathbf{h}_j + \mathbf{p}_j)$ yields the query and key vectors:

$$\mathbf{q}_i = Q(\mathbf{h}_i + \mathbf{p}_i) = \begin{bmatrix} \bar{\mathbf{p}}_i \\ \xi \langle \tilde{\mathbf{u}}_{\bar{\mathcal{T}}}, \bar{\mathbf{h}}_i \rangle \bar{\mathbf{p}}_1 \end{bmatrix}, \quad \mathbf{k}_j = K(\mathbf{h}_j + \mathbf{p}_j) = \begin{bmatrix} \eta \bar{\mathbf{p}}_{j+\ell} \\ \eta \bar{\mathbf{p}}_j \end{bmatrix}.$$

The pre-softmax attention logit is given by their inner product:

$$\langle \mathbf{q}_i, \mathbf{k}_j \rangle = \eta \langle \bar{\mathbf{p}}_i, \bar{\mathbf{p}}_{j+\ell} \rangle + \xi \langle \tilde{\mathbf{u}}_{\bar{\mathcal{T}}}, \bar{\mathbf{h}}_i \rangle \langle \bar{\mathbf{p}}_1, \bar{\mathbf{p}}_j \rangle. \quad (7)$$

Define the strict positive margins for positional separation and the BOS token:

$$\Delta_{\text{pos}} := \min_{i,j \neq i-\ell} \left(\langle \bar{\mathbf{p}}_i, \bar{\mathbf{p}}_i \rangle - \langle \bar{\mathbf{p}}_i, \bar{\mathbf{p}}_{j+\ell} \rangle \right) > 0,$$

$$\Delta_{\text{bos}} := \min_{j>1} \left(\langle \bar{\mathbf{p}}_1, \bar{\mathbf{p}}_1 \rangle - \langle \bar{\mathbf{p}}_1, \bar{\mathbf{p}}_j \rangle \right) > 0.$$

Case 1: $h_i \in \mathcal{T}$. By the orthonormal embedding assumption, $\tilde{\mathbf{h}}_i$ is orthogonal to all $\tilde{\mathbf{u}}_v$ for $v \notin \mathcal{T}$. Thus, $\langle \tilde{\mathbf{u}}_{\bar{\mathcal{T}}}, \tilde{\mathbf{h}}_i \rangle = 0$, and Equation 7 reduces to:

$$\langle \mathbf{q}_i, \mathbf{k}_j \rangle = \eta \langle \bar{\mathbf{p}}_i, \bar{\mathbf{p}}_{j+\ell} \rangle.$$

The target key $j = i - \ell$ achieves logit $\eta \langle \bar{\mathbf{p}}_i, \bar{\mathbf{p}}_i \rangle$, while any other $j \neq i - \ell$ is bounded above by $\eta (\langle \bar{\mathbf{p}}_i, \bar{\mathbf{p}}_i \rangle - \Delta_{\text{pos}})$. The logit gap is strictly $\Delta \geq \eta \Delta_{\text{pos}}$. By standard softmax concentration, choosing $\eta \geq \frac{1}{\Delta_{\text{pos}}} \log \frac{1}{\varepsilon}$ guarantees $s_{i,i-\ell} \geq 1 - \varepsilon$.

Case 2: $h_i \notin \mathcal{T}$. Here, $\tilde{\mathbf{h}}_i$ matches exactly one component of $\tilde{\mathbf{u}}_{\bar{\mathcal{T}}}$, yielding $\langle \tilde{\mathbf{u}}_{\bar{\mathcal{T}}}, \tilde{\mathbf{h}}_i \rangle = 1$. The logit becomes:

$$\langle \mathbf{q}_i, \mathbf{k}_j \rangle = \eta \langle \bar{\mathbf{p}}_i, \bar{\mathbf{p}}_{j+\ell} \rangle + \eta \xi \langle \bar{\mathbf{p}}_1, \bar{\mathbf{p}}_j \rangle.$$

We bound the difference between the BOS sink ($j = 1$) and any other position $j > 1$:

$$\langle \mathbf{q}_i, \mathbf{k}_1 \rangle - \langle \mathbf{q}_i, \mathbf{k}_j \rangle = \eta \xi \left(\langle \bar{\mathbf{p}}_1, \bar{\mathbf{p}}_1 \rangle - \langle \bar{\mathbf{p}}_1, \bar{\mathbf{p}}_j \rangle \right) - \eta \left(\langle \bar{\mathbf{p}}_i, \bar{\mathbf{p}}_{j+\ell} \rangle - \langle \bar{\mathbf{p}}_i, \bar{\mathbf{p}}_{1+\ell} \rangle \right).$$

The first term is lower-bounded by $\eta \xi \Delta_{\text{bos}}$. Using the trivial bound $|\langle \bar{\mathbf{p}}_i, \bar{\mathbf{p}}_{j+\ell} \rangle - \langle \bar{\mathbf{p}}_i, \bar{\mathbf{p}}_{1+\ell} \rangle| \leq 2d_{PE}$, the penalty is at most $2\eta d_{PE}$. Thus:

$$\langle \mathbf{q}_i, \mathbf{k}_1 \rangle - \langle \mathbf{q}_i, \mathbf{k}_j \rangle \geq \eta \xi \Delta_{\text{bos}} - 2\eta d_{PE}.$$

Choosing $\xi \geq \frac{3d_{PE}}{\Delta_{\text{bos}}}$ forces this gap to be at least ηd_{PE} . For sufficiently large η , the probability mass concentrates on the sink, ensuring $s_{i,1} \geq 1 - \varepsilon$.

In both cases, the target attention score exceeds $1 - \varepsilon$, completing the construction of $\text{FO}(\mathcal{T}, -\ell)$. $\square$

C.2 Full Construction for Theorem 4.1

We refer to $\text{id}(\mathbf{x})$ as the *identity subspace*, $\text{buf}_1(\mathbf{x}), \text{buf}_2(\mathbf{x})$ as the two *buffer subspaces*, and $\text{pos}(\mathbf{x})$ as the *positional encoding subspace*. We write $\mathbf{P}_B$ for the orthogonal projector onto block $B \in \{\text{id}, \text{buf}_1, \text{buf}_2\}$, $\mathbf{S}_{A \rightarrow B}$ for the rigid swap that copies block A into block B (zero elsewhere), and $\mathbf{\Pi}_\tau^B$ for the rank-1 projector onto the orthonormal axis $\mathbf{u}_\tau$ within block B (so e.g. $\mathbf{\Pi}_S^{\text{id}} = \sum_{s \in \mathcal{S}} \mathbf{u}_s \mathbf{u}_s^\top$ acting on id).

Building block: generalized attention chooser. We use a generalized fixed-offset attention mechanism extending Zhu et al. [2025c] (proven in Section C.1). We denote the operator by

$$\text{FO}(\mathcal{T}, -\ell),$$

which routes attention from tokens in $\mathcal{T}$ to the position ℓ steps earlier, and to BOS otherwise.

C.3 Layer 1: Workspace routing and context fetching

This layer routes state information, assigns phase tags, and retrieves context vectors corresponding to the current Q-table.

- **Head 1.1 (state $\rightarrow$ actions).**

$$\text{FO}(\mathcal{A}, -1), \quad W_V = \mathbf{P}_{\text{id}} \mathbf{\Pi}_S^{\text{id}}, \quad W_O = \mathbf{S}_{\text{id} \rightarrow \text{id}}.$$

Routes $\mathbf{u}_{s_t}$ into $\text{id}(a)$ for every action token.

- **Head 1.2 (state $\rightarrow$ Select).**

$$\text{FO}(\{\text{Select}\}, -2), \quad W_V = \mathbf{P}_{\text{id}} \mathbf{\Pi}_S^{\text{id}}, \quad W_O = \mathbf{S}_{\text{id} \rightarrow \text{id}}.$$

Routes $\mathbf{u}_{s_{t+1}}$ into $\text{id}(\text{Select})$.

- **Head 1.3 (state $\rightarrow$ reward).**

$$\text{FO}(\{r_t\}, -2), \quad W_V = \mathbf{P}_{\text{id}} \mathbf{\Pi}_S^{\text{id}}, \quad W_O = \mathbf{S}_{\text{id} \rightarrow \text{id}}.$$

Routes $\mathbf{u}_{s_t}$ into $\text{id}(r_t)$.

- **Head 1.4 (current-phase tagging).**

$$\text{FO}(\mathcal{A}, -2), \quad W_V = \mathbf{P}_{\text{id}} \mathbf{\Pi}_{Q_{\text{curr}}}^{\text{id}}, \quad W_O = \mathbf{I}.$$

Only a_t observes Q_{curr} at offset -2 , so only a_t receives the tag $\mathbf{u}_{Q_{\text{curr}}}$.

- **Head 1.5 (next-phase tagging).**

$$W_Q = \left(\mathbf{u}_{Q_{\text{next}}} \otimes \left(\sum_{a \in \mathcal{A}} \mathbf{u}_a \right) \right) \mathbf{P}_{\text{id}}, \quad W_K = \mathbf{\Pi}_{Q_{\text{next}}}^{\text{id}} \mathbf{P}_{\text{id}},$$

$$W_V = \mathbf{P}_{\text{id}} \mathbf{\Pi}_{Q_{\text{next}}}^{\text{id}}, \quad W_O = \mathbf{I}.$$

Each candidate action token attends to the unique Q_{next} marker in its causal past and receives $\mathbf{u}_{Q_{\text{next}}}$.

- **Head 1.6 (context fetch).**

$$W_Q = \left(\mathbf{\Pi}_{\mathcal{A}}^{\text{id}} + \left(\mathbf{u}_{\text{Update}} \otimes \sum_a \mathbf{u}_a \right) \right) \mathbf{P}_{\text{id}},$$

$$W_K = \left(\mathbf{\Pi}_{\mathcal{A}}^{\text{id}} + \mathbf{\Pi}_{\text{Update}}^{\text{id}} \right) \mathbf{P}_{\text{id}}, \quad W_V = \mathbf{P}_{\text{buf}_1}, \quad W_O = \mathbf{I}.$$

Each action token retrieves its context vector $\sum_s Q_t(s, a) \mathbf{u}_s$ into buf_1 .

Layer 1 output. Each action token stores the routed state in id and the corresponding Q-table column in buf_1 , while phase tags distinguish visited and candidate actions.

C.4 Layer 2: Q-value evaluation

This layer evaluates Q-values by contracting routed state vectors against retrieved context vectors.

- **Head 2.1 (Select fetches s_t).**

$$\begin{aligned} W_Q &= ((\mathbf{u}_{Q_{\text{curr}}} + \mathbf{u}_{\mathcal{A}}) \otimes \mathbf{u}_{\text{Select}}) \mathbf{P}_{\text{id}}, \\ W_K &= (\Pi_{Q_{\text{curr}}}^{\text{id}} + \Pi_{\mathcal{A}}^{\text{id}}) \mathbf{P}_{\text{id}}, \\ W_V &= \Pi_S^{\text{id}} \mathbf{P}_{\text{id}}, \quad W_O = \mathbf{S}_{\text{id} \rightarrow \text{buf}_1}. \end{aligned}$$

Select retrieves $\mathbf{u}_{s_t}$ from a_t .

- **Head 2.2 (Q-value computation).**

$$\begin{aligned} W_Q &= \Pi_S^{\text{id}}, \quad W_K = \mathbf{S}_{\text{buf}_1 \rightarrow \text{id}}, \\ W_V &= \Pi_{Q_{\text{next}}}^{\text{id}} - \Pi_{Q_{\text{curr}}}^{\text{id}}, \quad W_O = \mathbf{S}_{\text{id} \rightarrow \text{buf}_2}. \end{aligned}$$

The inner product computes $Q_t(s, a)$ and stages:

$$-Q_t(s_t, a_t) \mathbf{u}_{Q_{\text{curr}}} \quad \text{and} \quad Q_t(s_{t+1}, a) \mathbf{u}_{Q_{\text{next}}}$$

in buf_2 .

Layer 2 output. Current-state and next-state Q-values are stored in buf_2 on disjoint phase axes.

C.5 Layer 3: Action selection and maximization

- **Head 3.1 (action selection).**

$$\begin{aligned} W_Q &= (\mathbf{u}_{Q_{\text{next}}} \otimes \mathbf{u}_{\text{Select}}) \mathbf{P}_{\text{id}}, \\ W_K &= \beta \mathbf{S}_{\text{buf}_2 \rightarrow \text{id}} \Pi_{Q_{\text{next}}}^{\text{buf}_2}, \\ W_V &= \mathbf{P}_{\text{id}} \Pi_{\mathcal{A}}^{\text{id}}, \quad W_O = \mathbf{I}. \end{aligned}$$

Implements softmax action selection over $Q_t(s_{t+1}, a)$, recovering greedy selection as $\beta \rightarrow \infty$.

- **Head 3.2 (max extraction).** Uses the same query and key as Head 3.1, with

$$W_V = (\mathbf{u}_{\text{Select}} \otimes \mathbf{u}_{Q_{\text{next}}}) \mathbf{P}_{\text{buf}_2}, \quad W_O = \mathbf{S}_{\text{id} \rightarrow \text{buf}_2}.$$

Stores $\max_a Q_t(s_{t+1}, a)$ in $\text{buf}_2(\text{Select})$.

- **Head 3.3 (Update inheritance).**

$$\begin{aligned} W_Q &= (\mathbf{u}_{Q_{\text{curr}}} + \mathbf{u}_{\mathcal{A}}) \otimes \mathbf{u}_{\text{Update}} \mathbf{P}_{\text{id}}, \\ W_K &= (\Pi_{Q_{\text{curr}}}^{\text{id}} + \Pi_{\mathcal{A}}^{\text{id}}) \mathbf{P}_{\text{id}}, \\ W_V &= \Pi_{\mathcal{A}}^{\text{id}} + \mathbf{P}_{\text{buf}_1}, \quad W_O = \mathbf{I}. \end{aligned}$$

Update retrieves the visited action tag together with $Q_t(\cdot, a_t)$.

Layer 3 output. Select stores $(s_{t+1}, a^*, \max Q)$ while Update stores $(a_t, Q_t(\cdot, a_t))$.

C.6 Layer 4: TD error assembly

- **Head 4.1 (subtract current value).**

$$\begin{aligned} W_Q &= (\mathbf{u}_{Q_{\text{curr}}} \otimes \mathbf{u}_{\text{Update}}) \mathbf{P}_{\text{id}}, \\ W_K &= \mathbf{S}_{\text{buf}_2 \rightarrow \text{id}} \Pi_{Q_{\text{curr}}}^{\text{buf}_2}, \\ W_V &= \alpha \mathbf{S}_{\text{id} \rightarrow \text{buf}_1} \Pi_S^{\text{id}}, \quad W_O = \mathbf{I}. \end{aligned}$$

- **Head 4.2 (add reward).**

$$\begin{aligned} W_Q &= (\mathbf{u}_r \otimes \mathbf{u}_{\text{Update}}) \mathbf{P}_{\text{id}}, \\ W_K &= \mathbf{S}_{\text{buf}_1 \rightarrow \text{id}} \Pi_r^{\text{buf}_1}, \\ W_V &= \alpha \mathbf{S}_{\text{id} \rightarrow \text{buf}_1} \Pi_S^{\text{id}}, \quad W_O = \mathbf{I}. \end{aligned}$$

- **Head 4.3 (add discounted max).**

$$\begin{aligned} W_Q &= (\mathbf{u}_{\text{Select}} \otimes \mathbf{u}_{\text{Update}}) \mathbf{P}_{\text{id}}, \\ W_K &= \mathbf{S}_{\text{buf}_2 \rightarrow \text{id}} \Pi_{\text{Select}}^{\text{buf}_2}, \\ W_V &= \alpha \gamma \Pi_S^{\text{buf}_1}, \quad W_O = \mathbf{I}. \end{aligned}$$

Final output. Summing contributions in the residual stream yields

$$\sum_s Q_t(s, a_t) \mathbf{u}_s + \alpha (r_t + \gamma \max_a Q_t(s_{t+1}, a) - Q_t(s_t, a_t)) \mathbf{u}_{s_t},$$

which equals $\sum_s Q_{t+1}(s, a_t) \mathbf{u}_s$. This forms the updated context entry for the next step. $\square$

D Verification of Transformer Architecture

To validate that the proposed mechanisms implement the intended algorithms, we explicitly instantiated the constructions as handwired transformers with fixed weights matching the derived attention patterns. We then ran these models on the same inputs as their corresponding ground-truth algorithms and compared their internal states and outputs.

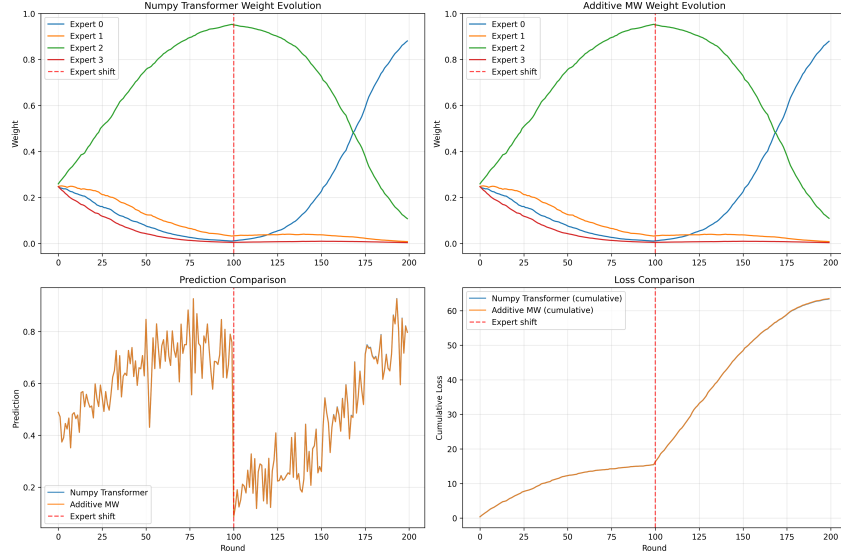


Figure 5: Multiplicative weights vs Transformer Expert weights

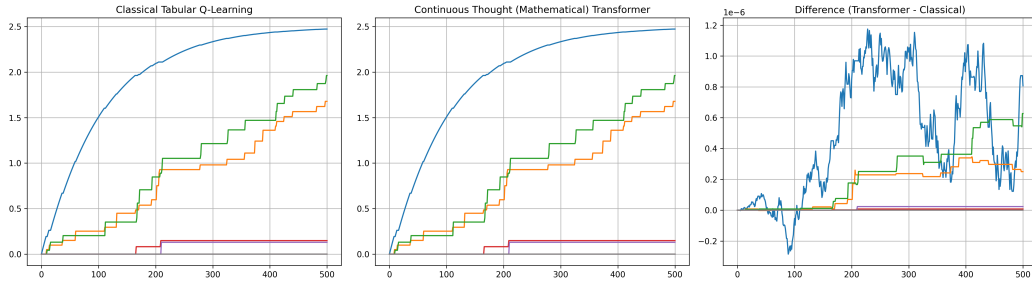


Figure 6: Tabular Q -learning vs Transformer Q -learning via COCONUT

Figures 5 and 6 show side-by-side comparisons between the handwired transformer and the ground-truth algorithms. In both cases, the trajectories are indistinguishable: the MWU construction assigns identical weights to experts over time, and the Q -learning construction produces Q -values that exactly match those of tabular Q -learning. These results provide direct empirical confirmation that the attention-based circuits derived in the proofs implement the desired online learning algorithms.

E Experimental Details

This section describes our experimental procedures and in-depth results for the unsupervised learning from expert-data, imitation learning of Q -learning, and LLM inference experiments. We provide our codebase in https://github.com/emiletimothy/transformer_decisionmaker.

E.1 Learning from Experts-Data Experiment

In this subsection, we describe our learning from experts experimental setup and results.

E.1.1 Experimental Setup Summary

1. **Model:** We train a GPT-2 style decoder [Radford et al., 2019] with AdamW, learning rate = 10^{-4} (cosine scheduler), weight decay = 10^{-2} , and gradient clipping at 1.0. Our dataset comprises of 3000 sequences, where each sequence length comprises of steps of length 100. In addition to the discrete tokens, every step is prefixed with a single continuous context token $m_t \in \mathbb{R}^d$ that acts as the models' persistent online state to store a superposition of the expert qualities (though we do not supervise this during training).
2. **Dataset:** The expert qualities are initiated uniformly at random between $[0.3, 0.9]$. At each step, there is a random binary true label, and each expert predicts the correct label with its probability (which is the quality), and gets a loss of 0 if correct and 1 if wrong. We train via a curriculum strategy where the losses are computed only at the masked positions, with loss $\mathcal{L} := \text{BCE}(\text{predicted decision}, \text{true label})$.
3. **Curriculum Strategy:** We teach the model to learn to reason over progressively larger MW sequences, one stage at a time. Specifically, for $1 \leq i \leq 10$, it trains on sequences truncated to $5i$ steps, and for $11 \leq i \leq 13$, it trains on sequences It trains on sequences truncated to $50 + 15(i - 10)$ steps. Every sequence is cut to these steps so the model doesn't see longer reasoning chains than what the current stage allows. Also, with probability 0.1, shorter sequences from previous stages are mixed into the current stage's dataset, preventing catastrophic forgetting.
4. **Training:** We train on 13 stages for up to 30 epochs/stage. Each epoch runs for 300 steps, and we set an early stopping after 5 stages if the loss did not decrease significantly (with a patience of 3 steps to prevent stochastic early stopping),
5. **Evaluation:** We evaluate the performance of our trained model in Figure 7 as well as in Figure 8 and Figure 9. Our results indicate that the latent context significantly improves the performance of the model on long synthetic prediction tasks, performing comparably to the optimal multiplicative weights update algorithm. Of note, when the quality of the best-performing experts becomes comparable, or when there is a correlation between the experts' predictions, the model's performance deteriorates.

E.1.2 Computational Resources

Each training job takes about 3 to 4 hours on an NVIDIA A100 80GB GPU.

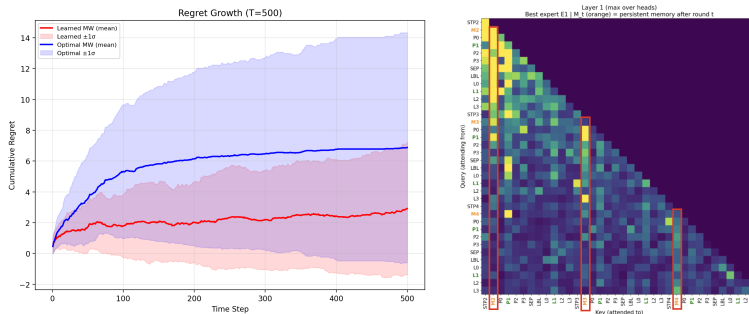


Figure 7: (Left) Cumulative regret plot, (Right) Causal attention heatmap on an MWU sequence

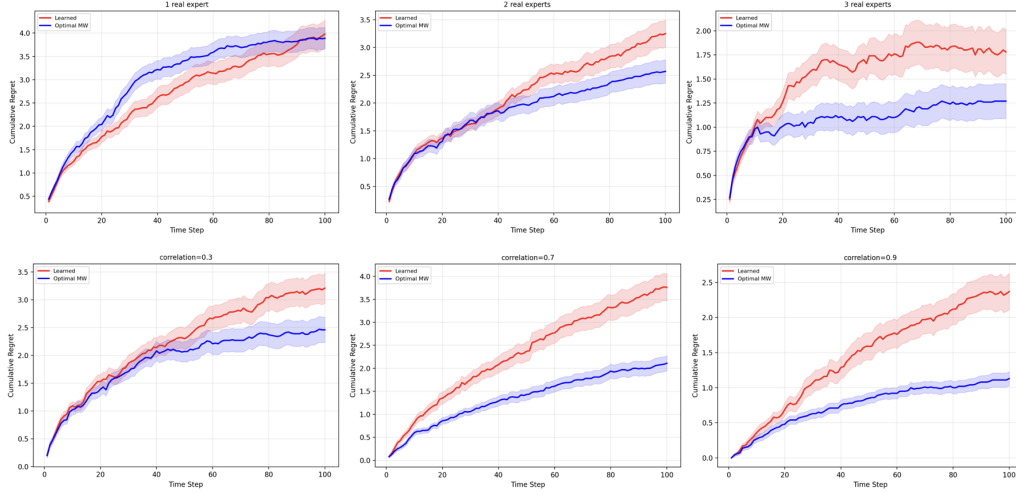


Figure 8: Evaluations on the transformer with latent context trained on expert data (without supervision) indicates that (top) the model’s performance to multiplicative weights is comparable when there is 1 and 2 high-quality experts ($p \geq 0.9$), but the model’s performance deteriorates with 3 high-quality experts, as it cannot significantly differentiate between the 3 high-quality predictions, and (bottom) the model also learns from its training data an inductive bias in the uniformity of the expert qualities. Thus, we see that the model’s performance deteriorates beyond this regime: as the correlation of the experts increases, the model’s performance decreases.

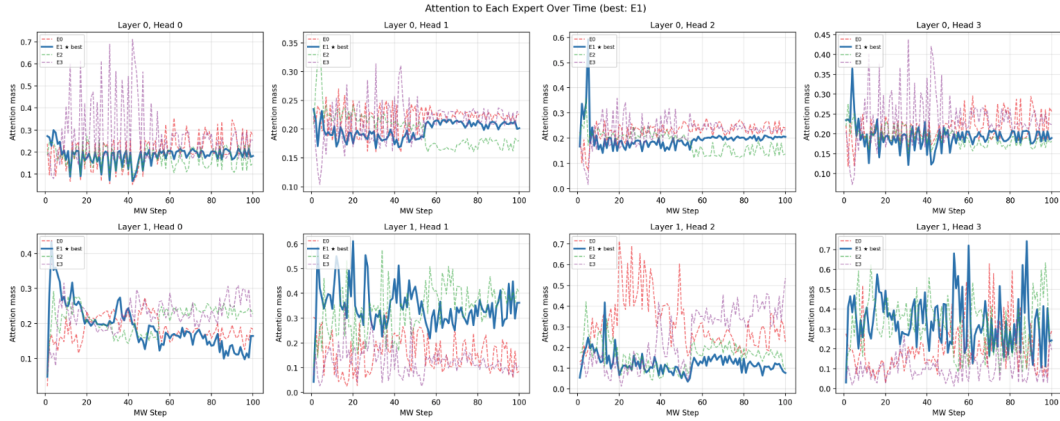


Figure 9: We plot the attention devoted to each expert across all the layers.

E.2 Tabular Q -learning Experiment

In this subsection, we describe our tabular Q -learning experimental setup and results.

E.2.1 MDP and Trajectory Sampling

Each training example is a single Q -learning episode on a freshly sampled finite MDP. For every episode we draw the state and action counts uniformly from configurable ranges, $|S| \sim \mathcal{U}\{S_{\min}, \dots, S_{\max}\}$ and $|\mathcal{A}| \sim \mathcal{U}\{A_{\min}, \dots, A_{\max}\}$, the episode length $T \sim \mathcal{U}\{T_{\min}, \dots, T_{\max}\}$, and the behavior policy’s exploration rate $\varepsilon \sim \mathcal{U}[0, 1]$. To avoid implicit biases in the reward and dynamics priors, every episode is assigned a (reward family, transition concentration) cell from the Cartesian product

reward $\in \{\text{peaked, bimodal, uniform, sparse, dense, Bernoulli}\}$,
transition $\in \{\text{near-deterministic, uniform, diffuse}\}$.

Cells are allocated in equal shares across the dataset and shuffled, giving balanced coverage. Given a cell, the transition kernel $P(\cdot \mid s, a) \sim \text{Dir}(\kappa \mathbf{1})$ is drawn with concentration $\kappa \in \{0.1, 1.0, 5.0\}$, and the reward table $R(s, a)$ is drawn from a Beta distribution whose shape parameters realize the named family (e.g. Beta(2, 2) for *peaked*); the *Bernoulli* family instead uses $R(s, a) \sim \mathcal{U}[0.1, 0.9]$ as the success probability.

Trajectory rollout. Starting from $s_0 \sim \mathcal{U}\{1, \dots, |\mathcal{S}|\}$ and $Q_0 \equiv 0$, we run T steps of tabular Q -learning under an ε -greedy behavior policy:

$$a_t \sim \begin{cases} \mathcal{U}(\mathcal{A}) & \text{w.p. } \varepsilon, \\ \arg \max_a Q_t(s_t, a) & \text{otherwise,} \end{cases} \quad (8)$$

$$r_t = R(s_t, a_t), \quad s_{t+1} \sim P(\cdot \mid s_t, a_t), \quad (9)$$

$$Q_{t+1}(s_t, a_t) = (1 - \alpha)Q_t(s_t, a_t) + \alpha(r_t + \gamma \max_{a'} Q_t(s_{t+1}, a')), \quad (10)$$

$$a_t^* = \arg \max_a Q_{t+1}(s_{t+1}, a). \quad (11)$$

We record the transition tuple $\tau_t = (s_t, a_t, r_t, s_{t+1}, a_t^*)$ together with a snapshot Q_{t+1} . After the rollout we apply independent random permutations $\pi_{\mathcal{S}}, \pi_{\mathcal{A}}$ to the state and action labels (and re-index each Q_{t+1} accordingly), so the model cannot exploit a canonical token ordering.

The dataset is split 90/10 into train and validation episodes and stored as a single tensor archive containing both splits and the generation config.

E.2.2 Per-Step Input Sequence

Tokenisation happens in the data collator at training time. Each transition τ_t is rendered into a length- $(2|\mathcal{A}| + 9)$ token sequence with two phases:

Phase 1 (predict reward, then a^*): BOS, Q_{curr} , s_t , a_t , r , Q_{next} ,
 (s_{t+1}, a_1) , (s_{t+1}, a_2) , $\dots$, $(s_{t+1}, a_{|\mathcal{A}|})$, SELECT,

Phase 2 (commit a^* , emit update): a_t^* , UPDATE.

The number of (s_{t+1}, a_c) pairs equals the MDP’s actual $|\mathcal{A}|$, not $A_{\max}$. The collator records the offsets of R, SELECT, and UPDATE so that supervision targets (r_t , a_t^* , and the recurrent state update) can be read off without re-parsing.

E.2.3 Continuous Context and Recurrence

In addition to the discrete tokens above, every step is prefixed with $|\mathcal{A}|$ continuous *context tokens* $c_1, \dots, c_{|\mathcal{A}|} \in \mathbb{R}^d$, one per action, that act as the model’s externalized Q-table. At the start of an episode the context is initialized from learned parameters, $C_0 = [c_1^{(0)}, \dots, c_{|\mathcal{A}|}^{(0)}]$. At step t , the transformer consumes C_t together with the discrete sequence above; the hidden state at the UPDATE position, h_t^{UPD} , replaces the row of C_t corresponding to the executed action a_t :

$$C_{t+1}[a, :] = \begin{cases} h_t^{\text{UPD}} & a = a_t, \\ C_t[a, :] & \text{otherwise.} \end{cases}$$

This recurrence is the only path along which information flows between steps; gradients propagate through the full episode unless a BPTT truncation window is set, in which case C is detached every K steps. Episodes are batched with a sampler that groups sequences sharing the same $|\mathcal{A}|$, so the context width is constant within a batch.

E.2.4 Experimental Setup Summary

1. **Model:** Pre-norm GPT-2 style decoder ($n_{\text{layers}} = 4$, $n_{\text{heads}} = 8$, $d_{\text{model}} = 256$, $d_{\text{ff}} = 1024$, dropout 0.1) with a recurrent continuous-context interface: $|A|$ learned context vectors $c_1, \dots, c_{|A|}$ are prepended to each step’s discrete token sequence:

$$[\text{BOS}, c_1, \dots, c_{|A|}, Q_{curr}, s_t, a_t, r_t, Q_{next}, (s_{t+1}, a_1)_{i=1}^{|A|}, \text{Select}, a^*, \text{Update}]$$

The hidden state at Update replaces c_{a_t} and is carried to step $t + 1$ (no cross-step attention, recurrence is the only channel for history). Optimizer: AdamW ($\beta = (0.9, 0.95)$), lr = $1e-4$ with linear warmup (500 steps) then cosine decay, weight decay = $1e-2$, gradient clipping at 1.0, mixed precision.

2. **Dataset:** 50,000 tabular Q -learning trajectories (45k train / 5k val). Each episode samples $n_S \sim \mathcal{U}(\{2, \dots, 8\})$, $n_A \sim \mathcal{U}(\{2, 3, 4\})$, $n_{\text{steps}} \sim \mathcal{U}(\{10, \dots, 50\})$, exploration $\varepsilon \sim \mathcal{U}(0, 1)$, with fixed $\alpha = 0.1, \gamma = 0.9$. MDPs are sampled across a grid of 6 reward distributions (peaked / bimodal / uniform / sparse / dense Beta, plus Bernoulli) $\times$ 3 transition concentrations (near-deterministic / uniform / diffuse Dirichlet). Random state and action permutations are applied per episode to enforce permutation invariance. Each transition stores (s, a, r, s', a^*) where $a^* = \arg \max_{a'} Q(s', a')$ from the running tabular Q -table.
3. **Curriculum / Loss Function:** Per-step cross-entropy on the SELECT logits against the tabular target a^* , masking phantom action slots when $|A| < \max |A|$:

$$\mathcal{L} := \frac{1}{T} \sum_{t=1}^T \text{CE}(\text{SELECT logits}_t, a_t^*).$$

Curriculum is over *action count*: training is split into 3 stages, each introducing one larger n_A . Stage k (epochs in $[\frac{(k-1)E}{3}, \frac{kE}{3})$) trains on episodes with $n_A \in \{2, \dots, 2+k\}$. Within a batch, all episodes share the same n_A (batch sampler grouping), so the recurrent context width is consistent.

4. **Training:** $E = 28$ epochs total, batch size 64, BPTT through full episodes with truncation window 10 (context detached every 10 steps to bound memory). Validation every 500 steps and at end of epoch; best-by-val-CE checkpoint is kept.

E.2.5 Additional Experimental Results

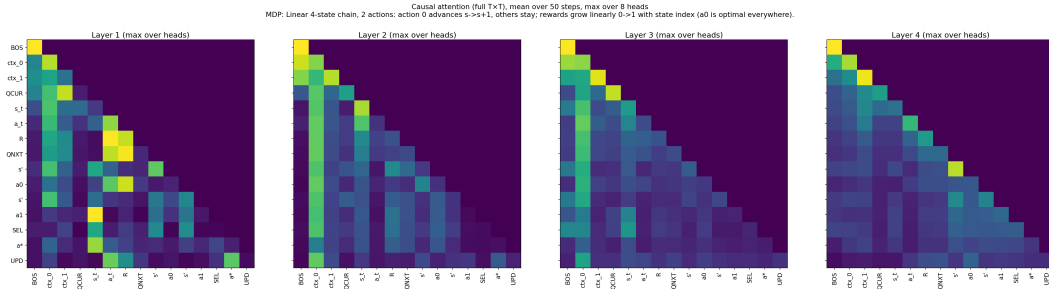


Figure 10: Full causal attention heatmap on a 4-state, 2-action linear-chain MDP where a_0 is optimal. The model concentrates attention on the context slot corresponding to a_0 , reflecting selection of the maximizing action. In addition, the Update token places strong attention on the visited action a_t , the reward token r_t , and the selected maximizing action a^* , precisely the operands required to form the TD error. These patterns provide direct evidence that the learned circuit performs context lookup, action maximization, and TD-style updates as in tabular Q -learning.

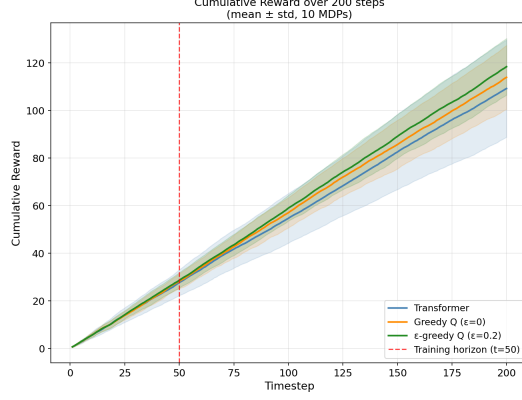


Figure 11: Long-horizon cumulative reward of learned transformer against tabular algorithm baselines. This demonstrates that the learned model continues to apply a stable update rule beyond the training horizon, maintaining performance comparable to tabular Q -learning and showing no degradation in the induced policy even under extended rollouts.

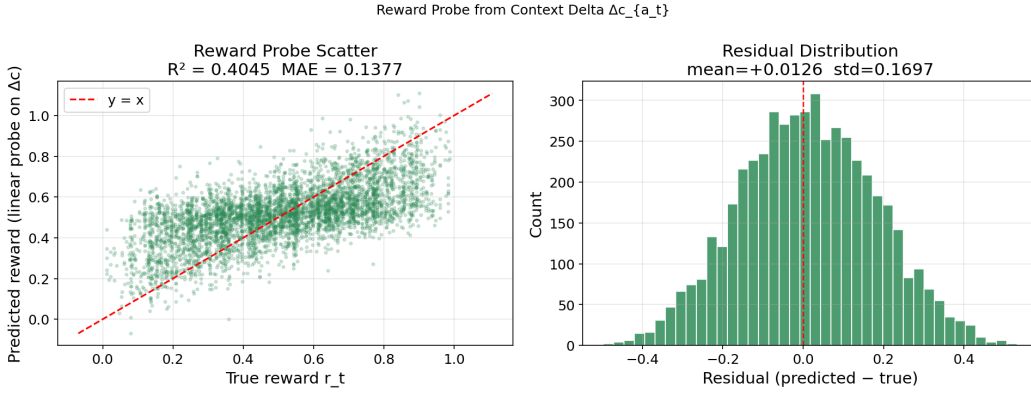


Figure 12: Linear probe of the reward signal from the per-step context update Δc_{a_t} . Left: predicted vs. true reward with the identity line. Right: residual distribution. The linear fit indicates that the model loosely encodes the reward explicitly in the update direction, consistent with the additive TD-error structure of Q -learning.

E.2.6 Computational Resources

Each Q -learning imitation training job takes about 3 to 4 hours on an NVIDIA A100 80GB GPU.

E.3 LLM inference experiments

This section provides the full experimental details for the LLM inference experiments in Section 5.3, including task generation, prompting protocols, full prompts, baselines, and additional results and detailed analyses of the notes generated by LLMs.

E.3.1 Experimental Setup

We evaluate frontier LLMs on interactive expert-prediction sequences. Our main models are DeepSeek-V3 (61 layers, 671B parameters) DeepSeek-AI et al. [2025] and Qwen-3-14B (40 layers, 14.8B parameters) [Yang et al., 2025], both evaluated without thinking modes. All experiments are inference-only, with no parameter updates. Each evaluation instance is a length-100 sequence with four fixed experts, Expert A, Expert B, Expert C, and Expert D. At each round, the model observes the four expert predictions and outputs its own prediction. The true outcome is then revealed as feedback before the next round.

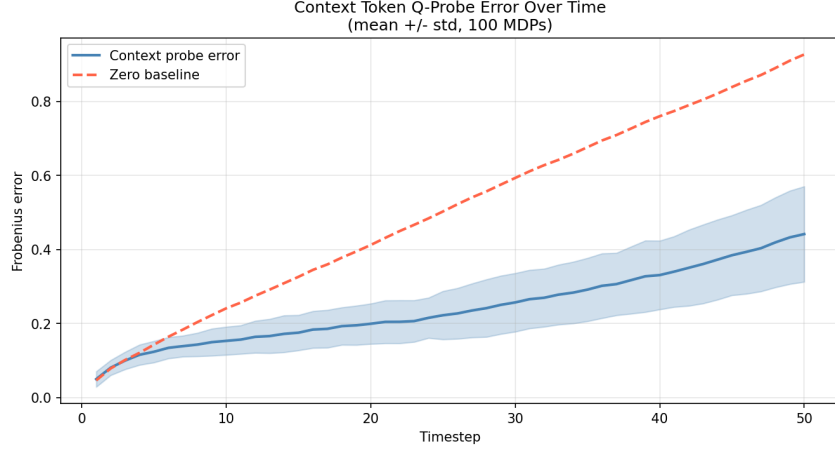


Figure 13: Error in reconstructing the tabular Q-values from the Update hidden state via a linear probe. Low reconstruction error across states indicates that the recurrent context tokens faithfully store a tabular Q-function, supporting the interpretation that the model implements Q -learning in its hidden state.

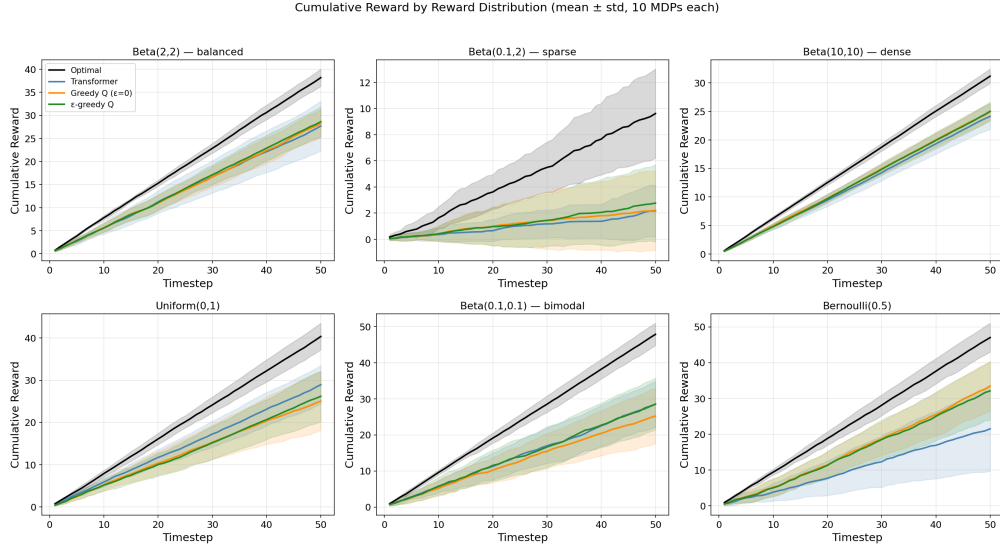


Figure 14: Cumulative reward over a 50-step horizon under six reward distributions, each averaged over 10 MDPs (mean $\pm$ std). Each panel fixes the family from which the per-(state, action) reward matrix $R[s, a]$ is drawn at MDP construction; rewards are then deterministic within an episode. We compare the Optimal policy within the 50-step horizon (black) against a Transformer (blue), Greedy Q -learning with $\alpha = 0$ (green), and ϵ -greedy Q -learning (orange). All learners track one another and trail the optimal policy by a roughly constant margin across continuous reward families (Beta and Uniform), with the largest absolute gap under sparse rewards (Beta(0.1, 2)). The Transformer notably underperforms both Q -learning baselines under Bernoulli(0.5): with rewards restricted to $\{0, 1\}$, the extreme value regime is strongly out of distribution. Additionally, many (s, a) pairs share identical immediate rewards, requiring bootstrapping through transitions to disambiguate actions — a regime that favors tabular Q -learning over the Transformer’s in-context value inference.

We use greedy decoding (temperature = 0) for DeepSeek-V3 and sampling with temperature = 0.7 and top- $p = 0.8$ for Qwen-3-14B, using the inference settings recommended for our API calls. Tools, code execution, and external retrieval are disabled. The model is instructed to output only valid JSON, with no explanations, intermediate calculations, or chain of thought. This setup lets us evaluate

online decision-making behavior directly, rather than the model’s ability to use tools or write out long reasoning steps.

Our main experiments use four prompting schemes, shown in Figure 3. These schemes are obtained by crossing two task framings with two state protocols.

The two task framings are:

- An **abstract online-learning framing**, where experts predict binary outcomes (0 or 1) and the task is described in terms of rounds and predictions.
- A **weather-forecasting framing**, where the same binary outcomes are presented as “sunny” or “rainy” and experts are described as weather forecasters.

The two state protocols are:

- **No-note protocol.** At each round, the expert predictions are sent as a user message, and the model responds only with its prediction. The true outcome is then appended as the next user message, with no model response to this feedback message. The full conversation history is retained across rounds, so the model has implicit access to all prior expert predictions, its own predictions, and the revealed outcomes through the growing context.
- **Note protocol.** At each round, the model first predicts from the expert predictions and the current note. After the true outcome is revealed, the model is asked to produce a short note containing information useful for future prediction. This note is then included in subsequent rounds alongside the conversation history. The note is unsupervised and unconstrained in format, and provides an explicit visible state channel on top of the raw interaction history.

Both protocols maintain the full conversation history. A history-free note variant is discussed in Section E.3.6 and Section E.3.7.

The note protocol tests whether an explicit, compressed state improves online adaptation beyond what the model can directly extract from raw history. The note is unsupervised: the model may write any information it finds useful, such as cumulative accuracy tallies for each expert. We do not constrain the note to follow any particular algorithmic format, so any useful structure in the note is self-discovered by the model.

For every run, we record the full interaction trace, including the prompt variant, expert predictions, model predictions, revealed outcomes, raw model responses, parsed JSON outputs, and notes when applicable. We report cumulative regret against the best fixed expert in hindsight on each prefix of the sequence.

E.3.2 Data Generation and Expert Regimes

Each evaluation instance consists of a length-100 sequence with 4 experts. For each instance, the true label at each round is drawn i.i.d. as $y_t \sim \text{Bernoulli}(0.5)$. Each expert i has a fixed accuracy $p_i \in [0, 1]$: at round t , expert i predicts y_t correctly with probability p_i and predicts $1 - y_t$ otherwise, independently across rounds and experts.

We construct three expert regimes by varying how expert accuracies are sampled. Each regime generates 30 instances.

Stratified. One expert is drawn from $\text{Uniform}(0.9, 1.0)$, one from $\text{Uniform}(0.65, 0.8)$, one from $\text{Uniform}(0.55, 0.7)$, and one from $\text{Uniform}(0.45, 0.6)$. This regime features a clearly dominant expert and tests whether the model can identify and follow it.

Flat. One expert is drawn from $\text{Uniform}(0.6, 0.7)$ and the remaining three from $\text{Uniform}(0.4, 0.6)$. All experts perform similarly, making it hard to distinguish the best expert from the rest. This regime tests the model’s ability to extract a weak signal from noisy feedback.

Anti-signal. One expert is drawn from $\text{Uniform}(0.6, 0.7)$, one adversarial expert from $\text{Uniform}(0.0, 0.1)$, and the remaining two from $\text{Uniform}(0.4, 0.6)$. The adversarial expert is correct less than 10% of the time, making it a strong *negative* signal: inverting its predictions would yield

accuracy above 90%. This regime tests whether the model can detect a systematically misleading expert and either discount it or use its predictions inversely.

For every instance, expert identities are randomly permuted after sampling the four accuracies.

E.3.3 Full Prompts

We provide the full prompts for the four prompting schemes used in the LLM experiments. Here we include the exact system instructions and input-output formats used for reproducibility.

Online with note.

You are helping in an online learning setting.

In each round, the same four experts - Expert A, Expert B, Expert C, and Expert D -

↪ make predictions about a binary outcome. Each expert predicts either 0 or 1.

↪ Some experts may be consistently more accurate than others.

Your job is to make your own prediction based on the experts' predictions. Your

↪ goal is to be as accurate as possible over time. Use past outcomes to figure

↪ out which experts are most trustworthy and improve your future predictions.

This interaction has two kinds of turns.

1. Prediction turn

In a prediction turn, you are given:

- * a short note from earlier rounds,

- * this round's predictions from the four experts.

You should use the note and the experts' predictions to make your prediction for

↪ this round.

Input format:

```
{
  "turn_type": "prediction",
  "note": "<short text>",
  "Expert_A": 0 or 1,
  "Expert_B": 0 or 1,
  "Expert_C": 0 or 1,
  "Expert_D": 0 or 1
}
```

Output format:

```
{
  "prediction": 0 or 1
}
```

2. Feedback turn

In a feedback turn, you are given:

- * the previous note,

- * this round's predictions from the four experts,

- * the true label for this round.

You should produce a new short note for future rounds. The note should keep only

↪ the information that is most useful for making later predictions.

Input format:

```
{
  "turn_type": "feedback",
  "note": "<short text>",
  "Expert_A": 0 or 1,
  "Expert_B": 0 or 1,
```

```
"Expert_C": 0 or 1,  
"Expert_D": 0 or 1,  
"true_label": 0 or 1  
}
```

Output format:

```
{  
"note": "<short text>"  
}
```

Important requirements:

- * Do not use any external tools.
- * Do not write or execute code.
- * Do not provide any explanation or reasoning steps.
- * Keep the note short and compact.
- * The note must be at most 500 words long.
- * The note should contain only the most useful information for future predictions.
- * Do not repeat the full history.
- * Output valid JSON only - no explanation, no reasoning steps, no other text.

Online without note.

You are helping in an online learning setting.

In each round, the same four experts - Expert A, Expert B, Expert C, and Expert D -

↪ make predictions about a binary outcome. Each expert predicts either 0 or 1.

↪ Some experts may be consistently more accurate than others.

Your job is to make your own prediction based on the experts' predictions. Your

↪ goal is to be as accurate as possible over time. Use past outcomes to figure

↪ out which experts are most trustworthy and improve your future predictions.

This interaction has two kinds of turns.

1. Prediction turn

In a prediction turn, you are given:

- * this round's predictions from the four experts.

You should use the experts' predictions to make your prediction for this round.

Input format:

```
{  
"turn_type": "prediction",  
"Expert_A": 0 or 1,  
"Expert_B": 0 or 1,  
"Expert_C": 0 or 1,  
"Expert_D": 0 or 1  
}
```

Output format:

```
{  
"prediction": 0 or 1  
}
```

2. Feedback turn

In a feedback turn, you are given:

- * the true label for this round.

You do not need to output anything for the feedback turn. Use this feedback when
↪ making later predictions.

Input format:

```
{  
  "turn_type": "feedback",  
  "true_label": 0 or 1  
}
```

Important requirements:

- * Do not use any external tools.
- * Do not write or execute code.
- * Do not provide any explanation or reasoning steps.
- * Output valid JSON only - no explanation, no reasoning steps, no other text.

Weather with note.

You are helping with a daily weather guess.

Each day, the same four weather experts - Expert A, Expert B, Expert C, and Expert D

- ↪ - make predictions about the weather. Each expert predicts either "sunny" or
- ↪ "rainy". They each have their own way of making forecasts, so some may be
- ↪ consistently more reliable than others.

Your job is to make your own prediction based on the experts' predictions. Your

- ↪ goal is to be as accurate as possible over time. Use past outcomes to figure
- ↪ out which experts are most trustworthy and improve your future predictions.

This interaction has two kinds of turns.

1. Forecast turn

In a forecast turn, you are given:

- * a short note from earlier days,
- * today's predictions from the four experts.

You should use the note and the experts' predictions to make your prediction for

- ↪ today.

Input format:

```
{  
  "turn_type": "forecast",  
  "note": "<short text>",  
  "Expert_A": "sunny" or "rainy",  
  "Expert_B": "sunny" or "rainy",  
  "Expert_C": "sunny" or "rainy",  
  "Expert_D": "sunny" or "rainy"  
}
```

Output format:

```
{  
  "prediction": "sunny" or "rainy"  
}
```

2. Feedback turn

In a feedback turn, you are given:

- * the previous note,
- * today's predictions from the four experts,
- * the actual weather for today.

You should produce a new short note for future days. The note should keep only the
↪ information that is most useful for making later guesses.

Input format:

```
{
  "turn_type": "feedback",
  "note": "<short text>",
  "Expert_A": "sunny" or "rainy",
  "Expert_B": "sunny" or "rainy",
  "Expert_C": "sunny" or "rainy",
  "Expert_D": "sunny" or "rainy",
  "actual_weather": "sunny" or "rainy"
}
```

Output format:

```
{
  "note": "<short text>"
}
```

Important requirements:

- * Do not use any external tools.
- * Do not write or execute code.
- * Do not provide any explanation or reasoning steps.
- * Keep the note short and compact.
- * The note must be at most 500 words long.
- * The note should contain only the most useful information for future predictions.
- * Do not repeat the full history.
- * Output valid JSON only - no explanation, no reasoning steps, no other text.

Weather without note.

You are helping with a daily weather guess.

Each day, the same four weather experts - Expert A, Expert B, Expert C, and Expert D
↪ - make predictions about the weather. Each expert predicts either "sunny" or
↪ "rainy". They each have their own way of making forecasts, so some may be
↪ consistently more reliable than others.

Your job is to make your own prediction based on the experts' predictions. Your
↪ goal is to be as accurate as possible over time. Use past outcomes to figure
↪ out which experts are most trustworthy and improve your future predictions.

This interaction has two kinds of turns.

1. Forecast turn

In a forecast turn, you are given:

- * today's predictions from the four experts.

You should use the experts' predictions to make your prediction for today.

Input format:

```
{
  "turn_type": "forecast",
  "Expert_A": "sunny" or "rainy",
  "Expert_B": "sunny" or "rainy",
  "Expert_C": "sunny" or "rainy",
  "Expert_D": "sunny" or "rainy"
}
```

Output format:

```
{
  "prediction": "sunny" or "rainy"
}
```


2. Feedback turn

In a feedback turn, you are given:

* the actual weather for today.

You do not need to output anything for the feedback turn. Use this feedback when
↪ making later guesses.

Input format:

```
{  
  "turn_type": "feedback",  
  "actual_weather": "sunny" or "rainy"  
}
```

Important requirements:

- * Do not use any external tools.
- * Do not write or execute code.
- * Do not provide any explanation or reasoning steps.
- * Output valid JSON only - no explanation, no reasoning steps, no other text.

E.3.4 Metrics and Baselines

Metrics. Our primary metric is **cumulative regret** against the best fixed expert in hindsight. Let $L_t = \sum_{s=1}^t \mathbf{1}[\hat{y}_s \neq y_s]$ denote the cumulative loss of a strategy and $L_t^{(i)} = \sum_{s=1}^t \mathbf{1}[\hat{y}_s^{(i)} \neq y_s]$ the cumulative loss of expert i . The regret at round t is

$$R_t = L_t - \min_{i \in [4]} L_t^{(i)}.$$

Regret measures how much worse a strategy performs compared to always following the single best expert. Lower regret indicates better adaptation to the reliable experts over the sequence. Each evaluation regime contains 30 independent instances. We report final regret at $T = 100$ averaged over these 30 instances. In regret curve plots, the shaded region shows ± 1 standard deviation across instances.

Baselines. We compare LLM performance against five reference strategies.

- **Multiplicative Weights (MW).** MW maintains a nonnegative weight $w_t^{(i)}$ for each expert, initialized uniformly, and predicts by weighted majority vote. After each round, weights are updated as $w_{t+1}^{(i)} = w_t^{(i)} \cdot \exp(-\eta \ell_t^{(i)})$ and renormalized, where $\ell_t^{(i)} \in \{0, 1\}$ is the loss of expert i at round t . The learning rate η is drawn uniformly from $[0.05, 0.5]$ per instance. MW serves as the classical long-memory online-learning reference strategy for tracking expert reliability over time.
- **Follow the Leader (FTL).** At each round, FTL predicts according to an expert with the lowest cumulative loss so far, breaking ties uniformly at random. This is a simple deterministic-history strategy that commits fully to the historically best expert.
- **Follow Previous Winners (FPW).** At each round, FPW predicts by majority vote among the experts who were correct in the previous round. If no expert is correct, or if there is a tie, it falls back to a uniform majority vote over all experts. This baseline uses only one round of feedback and therefore captures short-memory adaptation.
- **Majority Vote.** At each round, Majority Vote predicts by a uniform majority vote over all four experts, breaking ties uniformly at random. This is a non-adaptive baseline that ignores all feedback and treats experts as equally reliable.
- **Random Guessing.** Random Guessing predicts 0 or 1 uniformly at random at each round, independent of expert predictions and history. This provides a chance-level non-adaptive reference.

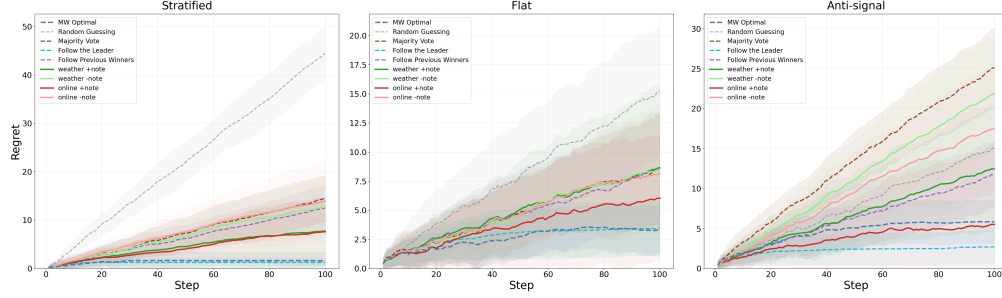


Figure 15: Cumulative regret of DeepSeek-V3 on three regimes. Regret is measured against the best fixed expert on each prefix. Note-based prompting reduces regret across the stratified, flat, and anti-signal regimes compared with no-note prompting and short-memory baselines.

These baselines span different levels of adaptation. MW is the classical long-memory online-learning baseline, FTL tracks the best cumulative expert, FPW uses only the previous round of feedback, and Majority Vote and Random Guessing are non-adaptive references. Comparing LLMs with FPW helps distinguish genuine long-horizon state tracking from short-term reactions to the most recent feedback, while comparison with MW measures how far the model remains from a standard online-learning strategy.

E.3.5 Main Results

We first present the main LLM results under the standard multi-turn protocol described in Section E.3.1. In this setting, the full interaction history is retained across rounds. We compare DeepSeek-V3 with and without an explicit note, using the same underlying expert sequences, task framings, and evaluation metrics. We focus on DeepSeek-V3 because its API provides effectively unlimited context, allowing it to complete length-100 runs with the full retained history. Qwen3-14B, whose context window is limited to 40,960 tokens, is evaluated under the history-free variant in Section E.3.6.

Table 1 reports the final cumulative regret of DeepSeek-V3 across all three expert regimes and both task framings. Figure 15 shows the corresponding regret trajectories over rounds.

Dataset	Weather		Online	
	+note	−note	+note	−note
Stratified	7.7 ± 7.0	12.7 ± 5.9	7.6 ± 7.0	13.9 ± 8.2
Flat	8.7 ± 7.1	8.5 ± 5.0	6.0 ± 5.4	8.2 ± 5.1
Anti-signal	12.5 ± 7.7	21.9 ± 7.0	5.0 ± 10.2	17.5 ± 7.6

Table 1: DeepSeek-V3 mean regret (± 1 std) under the multi-turn protocol with full history (30 instances, $T = 100$). Bold indicates conditions where the note protocol improves over no-note. Both protocols retain the full conversation history; the note protocol additionally asks the model to produce a state summary after each feedback turn.

Stratified regime. This is the regime shown in Figure 4 in the main text. The four expert accuracies are sampled from separated intervals, $[0.9, 1.0]$, $[0.65, 0.8]$, $[0.55, 0.7]$, and $[0.45, 0.6]$, and are then randomly assigned to Expert A, B, C, and D. Thus one expert is clearly dominant, making this the cleanest setting for testing whether the model can identify and exploit a reliable source from feedback. As discussed in Section 5.3, the regret curves of the no-note prompts remain close to simple short-memory baselines, while the note prompts achieve substantially lower regret. This suggests that the explicit note helps the model preserve cumulative expert-reliability information across rounds.

The note traces provide one explanation for this improvement. The model is never instructed to use any particular note format, yet in the weather framing it often adopts a cumulative-counter strategy within the first few steps. At step 0, the note is a qualitative description, such as B,C,D correct; A wrong. By step 1, it transitions to a running tally, such as B,C,D correct 2/2; A

wrong 2/2; by step 3, it stabilizes into a compact counter, such as A 1/4; B 3/4; C 3/4; D 3/4. About 60% of instances adopt this counter format by the first feedback turn, rising to 70% by step 5. By step 99, notes such as A:45/100, B:94/100, C:69/100, D:77/100 make the dominant expert immediately apparent. This format effectively stores the sufficient statistics needed by a Follow-the-Leader-style strategy, which explains why the note curves approach that baseline.

In the online framing, the model instead tends to record a round-by-round transcript, for example Round1: A=1,B=0,C=0,D=0, label=0. A wrong, others correct. Each step appends a new entry, and by step 10 the note begins compressing into pattern summaries, such as A wrong when predicts 1 (rounds 1,4,7). This round-listing format grows linearly and reaches more than 90,000 context tokens by step 100 in the retained-history protocol. Nevertheless, it does not prevent the model from extracting the relevant signal when the full interaction history is also available.

Flat regime. In the flat regime, the best expert is only weakly better than the others, so the feedback signal is noisy and difficult to identify from short prefixes. Three of the four LLM conditions, weather +note, weather −note, and online −note, cluster near the FOLLOW PREVIOUS WINNERS and MAJORITY VOTE baselines. In particular, the note provides little benefit in the weather framing. The exception is online +note, which stays noticeably below the other LLM curves throughout the sequence. This suggests that, even in a low-signal environment, the combination of online framing and an explicit note can extract a weak but consistent advantage.

The note behavior helps explain the difference between regimes. About 80% of flat-regime instances converge to cumulative counters by step 5, similar to the stratified regime. Thus, the note format itself is often reasonable. However, the expert accuracy differences are small enough that a tally such as A:65/100, B:54/100 may still not yield a confident ranking. The counter faithfully records the evidence, but the evidence itself remains ambiguous. In the online framing, the round-listing note preserves finer-grained information about individual outcomes. Together with the retained interaction history, this appears to give the model additional structure for acting on small accuracy differences that compact weather counters may miss.

Anti-signal regime. The anti-signal regime produces the largest separation between the note and no-note conditions. Since one expert has accuracy between 0 and 10%, MAJORITY VOTE is systematically hurt by this adversarial source and falls below RANDOM GUESSING. Without a note, the LLM performs between RANDOM GUESSING and MAJORITY VOTE, suggesting that it partially avoids the misleading expert but does not maintain a sufficiently stable estimate of expert reliability. Adding a note substantially changes the behavior. The note curves flatten, and in the online framing the note curve comes closest to the MW baseline among the LLM conditions. This indicates that the note helps the model preserve enough reliability information to identify which experts should be trusted and which should be ignored.

The mechanism is visible in the note evolution. By step 5, the cumulative counter already reveals the adversarial expert’s near-zero accuracy, for example B:1/6, while the best expert begins to stand out, for example C:4/6. By step 10, the signal is clearer, as in A:5/11, B:1/11, C:7/11, D:4/11; by step 50, it is unambiguous, as in A:19/51, B:3/51, C:31/51, D:24/51. The model uses this information primarily for exclusion, following the best expert while ignoring the adversarial one. Across 30 instances, the final note correctly identifies the adversarial expert as the worst in 60% of parseable cases. However, the visible notes in our runs do not explicitly describe an inversion strategy, namely, predicting the opposite of the adversarial expert.

E.3.6 History-Free Note Variant and Cross-Model Comparison

We next evaluate a *history-free variant* of the note protocol for cross-model comparison. This variant modifies only the note protocol: the raw conversation history is not carried across rounds, and the note is the model’s only explicit state. Each round consists of two independent API calls. In the prediction call, the model receives the system prompt, the note from the previous round, and the current expert predictions, and then outputs its prediction. In the feedback call, the model receives the same inputs together with the revealed outcome, and outputs an updated note.

For the −note baseline, we keep the standard multi-turn no-note protocol used throughout the paper: the model outputs only predictions, the true outcomes are appended to the conversation history, and the full interaction history is retained across rounds. Thus, the comparison asks whether a

model-generated note, used as the only compressed state, can match or improve upon the model’s ability to use raw conversation history without an explicit note.

This variant is useful for two reasons. First, it directly tests whether a natural-language note can serve as a compressed online-learning state. Second, it enables matched comparisons between models with different context-length capabilities. In the standard multi-turn note protocol, the context grows linearly with the number of rounds because every previous prediction turn, feedback turn, and note update remains in the conversation history. In our implementation, at $T = 100$, the conversation history reaches approximately 20,000 tokens for the weather framing and more than 90,000 tokens for the online framing. DeepSeek-V3 can complete these length-100 retained-history runs in our setup, but Qwen-3-14B is limited to 40,960 tokens by its positional encoding and exhausts this budget around rounds 42 to 50. We therefore use the history-free variant for matched cross-model comparisons over the full length-100 horizon.

Table 2 compares DeepSeek-V3 and Qwen-3-14B under the history-free variant. This comparison does not test whether a note improves performance on top of raw history, as in Section E.3.5. Instead, it tests whether different LLMs can use a self-generated note as the *only* cross-round state for online prediction.

Dataset	Model	Weather		Online	
		+note	−note	+note	−note
Stratified	DeepSeek-V3	8.1 ± 6.3	12.7 ± 5.9	9.3 ± 8.0	13.9 ± 8.2
	Qwen-3-14B	32.0 ± 8.9	13.1 ± 6.3	20.7 ± 15.3	17.8 ± 7.6
Flat	DeepSeek-V3	7.4 ± 5.2	8.5 ± 5.0	7.9 ± 5.3	8.2 ± 5.1
	Qwen-3-14B	12.0 ± 5.3	8.6 ± 4.3	10.4 ± 9.4	9.7 ± 4.7
Anti-signal	DeepSeek-V3	13.3 ± 6.9	21.9 ± 7.0	11.3 ± 10.3	17.5 ± 7.6
	Qwen-3-14B	19.3 ± 9.9	19.5 ± 5.5	20.3 ± 17.7	16.5 ± 9.1

Table 2: Mean regret (± 1 std) over 30 instances of length $T = 100$. The +note columns use the history-free note variant, where the note is the only cross-round state. The −note columns use the standard multi-turn no-note protocol, where the full interaction history is retained but no explicit note is provided.

The two models exhibit qualitatively opposite responses to the note mechanism.

DeepSeek-V3. The note consistently reduces regret across all regimes. DeepSeek-V3 often uses a cumulative-counter format for its notes, such as A: 45/100, B: 94/100, C: 69/100, D: 77/100, effectively maintaining a running tally of accuracy. Under the history-free variant, where the note is the only cross-round state, this counter serves as a useful statistic for identifying reliable experts.

Qwen-3-14B. In contrast, the note hurts performance for Qwen-3-14B in all six comparisons in Table 2. Across the Qwen-3-14B note runs in this table, the model never produces cumulative statistics. Its notes describe only the most recent feedback, for example, *Expert A was correct, Experts B, C, and D were incorrect*. Such notes overwrite rather than accumulate past information. Since the history-free variant provides no raw conversation history, Qwen-3-14B effectively has only one round of memory despite having access to the note channel. As a result, the note can mislead the model into overreacting to the most recent feedback.

This comparison shows that the ability to use an external scratchpad for online learning is not a generic consequence of giving an LLM a note field. It depends on whether the model can discover an effective compression strategy for the relevant online state. DeepSeek-V3 discovers a cumulative-counting strategy without being instructed to do so, while Qwen-3-14B does not. This difference may be related to differences in model scale and training, but our experiment does not isolate the cause. Understanding which model properties enable such self-discovered compression strategies is an interesting direction for future work.

E.3.7 Ablation: History Retention and Note Format

We next compare two ways of using the note with DeepSeek-V3. In the standard multi-turn note protocol, the model receives both the retained conversation history and the latest note. In the history-free note variant, the model receives only the latest note and the current expert predictions, so the note is the sole cross-round state. This ablation asks whether the natural-language note is sufficient as a compressed state, and what additional role is played by the retained raw history.

Comparing the +note entries across Table 1 and Table 2, we find that most conditions are similar between the two settings. The main exception is the anti-signal regime under the online framing: retaining history reduces regret from 11.3 to 5.0, a gap of 6.3. The behavior is consistent with an implicit inversion strategy: with full history, the model can observe that the anti-signal expert is almost always wrong and use this pattern to improve prediction. However, this strategy is not explicitly described in the written note.

Note format summary. Table 3 summarizes the note formats adopted by DeepSeek-V3 at step 99 across all conditions, for both the retained-history and history-free settings. The contrast between settings and framings is striking.

Setting	Dataset	Framing	Note format at step 99 (out of 30)				
			Counter	Round list	Cond. ctr	Last-step	Other
Full history	Stratified	weather	21	0	1	8	0
		online	8	18	0	2	2
	Flat	weather	24	0	0	5	1
		online	8	19	0	3	0
	Anti-signal	weather	23	0	0	6	1
		online	10	17	0	3	0
History-free	Stratified	weather	16	0	10	2	2
		online	21	1	0	7	1
	Flat	weather	21	0	7	0	2
		online	27	1	0	2	0
	Anti-signal	weather	15	0	7	6	2
		online	23	2	0	1	4

Table 3: DeepSeek-V3 note format at step 99 over 30 instances per condition. “Counter” is a cumulative accuracy tally, e.g., A:45/100, B:94/100. “Round list” is an appended per-round transcript or compressed pattern summary, e.g., Round1: A=1,B=0,C=0,D=0,label=0; A wrong, others correct. Round2: all 0,label=0; all correct. “Cond. ctr” splits accuracy by weather outcome, e.g., A: sunny 22/39, rainy 18/61. “Last-step” records only the latest feedback, e.g., A correct; B,C,D wrong. “Other” includes vague, mixed-format, or malformed notes. Full history denotes the retained-conversation protocol; history-free denotes the note-only variant.

The table shows that note format is primarily driven by prompt framing and the availability of history. With retained history, weather prompts mostly produce cumulative counters, whereas online prompts mostly produce round-by-round transcripts. Once the raw history is removed, online prompts also shift toward counters. This suggests that round listing is used mainly when the conversation history can serve as a fallback; when the note is the only cross-round state, the model is pushed toward a compressed statistic.

The weather framing also reveals a history-free failure mode: in roughly a quarter of final notes, the model writes conditional counters that split expert accuracy by weather outcome instead of maintaining a single overall reliability estimate. These notes preserve partial information, but make the relevant expert ranking less directly available than a standard counter. This failure mode nearly disappears when the raw history is retained. Overall, the same patterns appear across stratified, flat, and anti-signal regimes, suggesting that note format is governed more by framing and memory constraints than by the difficulty of the expert sequence.